



Real Time Systems - rtos

Electronics and communication engineering (Jawaharlal Nehru Technological University,
Hyderabad)



Scan to open on Studocu

JNTU HYDERABAD**REAL TIME SYSTEMS****B.TECH./CSE & IT/R18**
.....**SYLLABUS****UNIT – I**

Introduction: Introduction to UNIX/LINUX, Overview of Commands, File I/O (open, create, close, lseek, read, write), Process Control (fork, vfork, exit, wait, waitpid, exec).

UNIT – II

Real Time Operating Systems: Brief History of OS, Defining RTOS, The Scheduler, Objects, Services, Characteristics of RTOS, Defining a Task, asks States and Scheduling, Task Operations, Structure, Synchronization, Communication and Concurrency. Defining Semaphores, Operations and Use, Defining Message Queue, States, Content, Storage, Operations and Use.

UNIT – III

Objects, Services and I/O: Pipes, Event Registers, Signals, Other Building Blocks, Component Configuration, Basic I/O Concepts, I/O Subsystem.

UNIT – IV

Exceptions, Interrupts and Timers: Exceptions, Interrupts, Applications, Processing of Exceptions and Spurious Interrupts, Real Time Clocks, Programmable Timers, Timer Interrupt Service Routines (ISR), Soft Timers, Operations.

UNIT – V

Case Studies of RTOS: RT Linux, MicroC/OS-II, Vx Works, Embedded Linux, and Tiny OS.

UNIT – I

INTRODUCTION

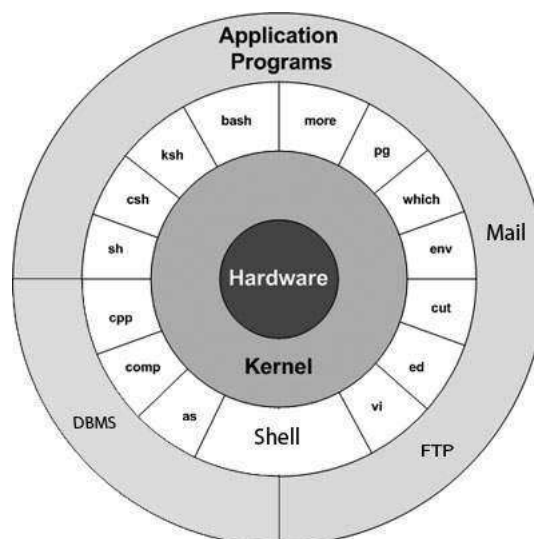
INTRODUCTION TO UNIX/LINUX

The Unix operating system is a set of programs that act as a link between the computer and the user. The computer program that allocates the system resources and coordinates all the details of the computer's internals is called the operating system or the kernel.

Users communicate with the kernel through a program known as the shell. The shell is a command line interpreter; it translates commands entered by the user and converts them into a language that is understood by the kernel.

- ⇒ UNIX was originally developed in 1969 by a group of AT&T employees Ken Thompson, Dennis Ritchie, Douglas McIlroy, and Joe Ossanna at Bell Labs.
- ⇒ There are various UNIX variants available in the market. Solaris UNIX, AIX, HP UNIX and BSD are a few examples. Linux is also a flavor of UNIX which is freely available.
- ⇒ Several people can use a UNIX computer at the same time; hence UNIX is called a multiuser system.
- ⇒ A user can also run multiple programs at the same time; hence UNIX is a multitasking environment.

UNIX Architecture: Here is a basic block diagram of a Unix system:



The main concept that unites all the versions of Unix is the following four basics:

- 1) Kernel:** The kernel is the heart of the operating system. It interacts with the hardware and most of the tasks like memory management, task scheduling and file management.
- 2) Shell:** The shell is the utility that processes your requests. When you type in a command at your terminal, the shell interprets the command and calls the program that you want. The shell uses standard syntax for all commands. C Shell, Bourne Shell and Korn Shell are the most famous shells which are available with most of the Unix variants.
- 3) Commands and Utilities:** There are various commands and utilities which you can make use of in your day to day activities. cp, mv, cat and grep, etc. are few examples of commands and utilities. There are over 250 standard commands plus numerous others provided through 3rd party software. All the commands come along with various options.
- 4) Files and Directories:** All the data of Unix is organized into files. All files are then organized into directories. These directories are further organized into a tree-like structure called the filesystem.

OVERVIEW OF COMMANDS

1) cd (change directory): This command is used to change the current working directory. Here are some examples of using cd:

```
cd datafiles      cd ..      cd /      cd $HOME
```

2) mkdir (make directory): This command makes a sub-directory under the current working directory.

```
mkdir junk
```

3) rmdir (remove directory): This command removes (deletes) a sub-directory under the current working directory. The directory to be removed must be empty of all files and sub-directories.

```
rmdir junk
```

4) ls (list files): This command is similar to DIR in DOS; it displays a list of all the files in the directory. New users don't have any files in their home directory. Here is an example of using this command;

```
ls
```

You will note that the shell prompt reappears, and there is no file listing of the directory contents. This does not mean that there are no files stored there. Just like DOS, UNIX systems support hidden files.

Here is an example of using the command with switches:

```
ls -la
```

The switch l stands for long listing, and the a switch is for all files, including directories and hidden files. UNIX responds with the following listing of the directory

```
ls -la
total 6    -rw-rw-r-- 1 jmsmith  staff 526 Apr 15 11:03 myletter
```

5) cat (concatenate files): This command is used to combine files or to print files to the screen. By default, the cat command sends its output to your screen (in UNIX we call this standard-output or stdout for short).

The following command can be used to view the file, ".profile" on stdout:

```
cat .profile
```

The format of this command specifies that cat will use the file, ".profile" as its input, and send the output to your screen.

Note: if the file is very large or is not a plain text file, cat will try to print it to the screen anyway, sometimes with undesirable results. The less command is better suited for viewing files than the cat command.

The cat utility can also be used with the unix shell's output redirection operators to join many files together into a single file. The command:

```
cat temp .profile > temp1
```

copies the two files (temp and .profile) to a file called temp1.

The >> is used for appending to the end of an existing file, ie:

```
cat temp >> temp1
```

6) cp (copy files): This command stands for copy, and is used for copying one file to another.

Example:

```
cp .profile temp2
```

This copies the file .profile to another called temp2. If temp2 had already existed, its previous contents would've been erased.

Files can also be copied to another directory. The command

```
cp * /usr/tmp
```

would copy all the files in the current directory to the directory /usr/tmp.

7) mv (move files): The mv command is used for moving or renaming files.

Example:

```
mv temp temp2
```

This renames the file temp to temp2.

As an example (do not type this!), the command:

```
mv temp2 /tmp
```

would move the file temp2 into the directory /tmp (it would no longer appear in your home directory).

8) rm (remove files): The rm utility is used for erasing files and directories.

Example:

```
rm temp2
```

This removes the file. Once a file is removed, it cannot be restored. To cover situations where mistakes might occur, a switch -i appended to this command will request a Yes or No response before deleting the file.

Example:

```
rm -i temp1
```

Other Basic Commands:

9) who (display a list of users): This command will display users currently on the system.

```
who
```

10) help (obtain help on command usage): Most commands provide a short summary of their switches and arguments when you give them the "--help" switch.

Example: To obtain help for the command grep type:

```
grep --help
```

11) bg (run programs in the background): This command allows you to run a program in the background while you perform other tasks.

```
bg <name of job>
```

```
fg <name of job>
```

brings it back to the foreground.

12) less (page through a text file): This command allows you to view a text file without fear of accidentally modifying it, as you would with a text editor.

Example:

```
less /etc/hosts
```

FILE I/O (OPEN, CREATE, CLOSE, LSEEK, READ, WRITE)

File Descriptors: To the kernel, all open files are referred to by file descriptors. A file descriptor is a non-negative integer. By convention, UNIX System shells associate file descriptor 0 with the standard input of a process, file descriptor 1 with the standard output, and file descriptor 2 with the standard error.

Although their values are standardized by POSIX.1, the magic numbers 0, 1, and 2 should be replaced in POSIX-compliant applications with the symbolic constants `STDIN_FILENO`, `STDOUT_FILENO`, and `STDERR_FILENO` to improve readability. These constants are defined in the `<unistd.h>` header.

1) open and openat Functions:

```
#include <fcntl.h> /* Both return: file descriptor if OK, -1 on error */
int open(const char *path, int oflag, ... /* mode_t mode */);
int openat(int fd, const char *path, int oflag, ... /* mode_t mode */);
```

This argument is formed by **OR** together one or more of the following constants from the `<fcntl.h>` header:

- i. **O_RDONLY:** Open for reading only.
- ii. **O_WRONLY:** Open for writing only.
- iii. **O_RDWR:** Open for reading and writing.
- iv. **O_EXEC:** Open for execute only.
- v. **O_APPEND:** Append to the end of file on each write.
- vi. **O_CREAT:** Create the file if it doesn't exist.

This option requires a third argument to the open function — the mode, which specifies the access permission bits of the new file.

- i. **O_NONBLOCK:** Set the nonblocking mode for FIFO, block special file, or character special file.
- ii. **O_SYNC:** Have each write wait for physical I/O to complete.
- iii. **O_TRUNC:** If the file exists and if it is successfully opened for either write-only or read-write, truncate its length to 0

2) creat Function: A new file can also be created by calling the creat function.

```
#include <fcntl.h> /* Returns: file descriptor opened for write-only if OK, -1 on error */
int creat(const char *path, mode_t mode);
```

Note that this function is equivalent to `open(path, O_WRONLY | O_CREAT | O_TRUNC, mode)`

3) close Function: An open file is closed by calling the close function.

```
#include <unistd.h> /* Returns: 0 if OK, -1 on error */
int close(int fd);
```

Closing a file also releases any record locks that the process may have on the file.

4) lseek Function: Every open file has an associated “current file offset”, normally a non-negative integer that measures the number of bytes from the beginning of the file (certain devices could allow negative offsets). By default, this offset is initialized to 0 when a file is opened, unless the `O_APPEND` option is specified.

```
#include <unistd.h> /* Returns: new file offset if OK, -1 on error */
off_t lseek(int fd, off_t offset, int whence);
```

The interpretation of the offset depends on the value of the whence argument:

- i. **SEEK_SET:** set to offset bytes from the beginning of the file.
- ii. **SEEK_CUR:** set to its current value plus the offset.
- iii. **SEEK_END:** set to the size of the file plus the offset.

We can seek zero bytes from the current position to determine the current offset:

```
off_t currpos; currpos = lseek(fd, 0, SEEK_CUR);
```

The file’s offset can be greater than the file’s current size, in which case the next write to the file will extend the file. This is referred to as creating a hole in a file and is allowed. Any bytes in a file that have not been written are read back as 0 (can be checked by `od(1)` command).

5) read Function:

```
#include <unistd.h> /* Returns: number of bytes read, 0 if end of file, -1 on error */ ssize_t read(int fd, void *buf, size_t nbytes);
```

If the read is successful, the number of bytes read is returned. If the end of file is encountered, 0 is returned. The read operation starts at the file's current offset. Before a successful return, the offset is incremented by the number of bytes actually read.

6) write Function:

```
#include <unistd.h> /* Returns: number of bytes written if OK, -1 on error */ ssize_t write(int fd, const void *buf, size_t nbytes);
```

The return value is usually equal to the nbytes argument; otherwise, an error has occurred. A common cause for a write error is either filling up a disk or exceeding the file size limit for a given process. For a regular file, the write operation starts at the file's current offset. If the O_APPEND option was specified when the file was opened, the file's offset is set to the current end of file before each write operation. After a successful write, the file's offset is incremented by the number of bytes actually written.

PROCESS CONTROL (FORK, VFORK, EXIT, WAIT, WAITPID, EXEC)

1) fork() : System call to create a child process.

This will yield output mentioning what is fork used for, syntax and along with all the required details.

The syntax used for the fork system call is as below,

```
pid_t fork(void);
```

Fork system call creates a child that differs from its parent process only in pid(process ID) and ppid(parent process ID). Resource utilization is set to zero. File locks and pending signals are not inherited. (In Linux "fork" is implemented as "copy-on-write").

Note: "Copy on write" -> Whenever a fork() system call is called, a copy of all the pages(memory) related to the parent process is created and loaded into a

separate memory location by the Operating System for the child process. But this is not needed in all cases and may be required only when some process writes to this address space or memory area, then only separate copy is created/provided.

2) **vfork()** :

vfork -> create a child process and block parent process.

Note: In vfork, signal handlers are inherited but not shared.

```
shashi@linuxtechi ~}$ man vfork
```

This will yield output mentioning what is vfork used for, syntax and along with all the required details.

```
pid_t vfork(void);
```

vfork is as same as fork except that behavior is undefined if process created by vfork either modifies any data other than a variable of type pid_t used to store the return value p of vfork or calls any other function between calling _exit() or one of the exec() family.

Note: vfork is sometimes referred to as special case of clone.

3) wait(): wait() system call suspends execution of current process until a child has exited or until a signal has delivered whose action is to terminate the current process or call signal handler.

```
pid_t wait(int * status);
```

There are other system calls related to wait as below:

- i. waitpid():** suspends execution of current process until a child as specified by pid arguments has exited or until a signal is delivered.

```
pid_t waitpid (pid_t pid, int *status, int options);
```
- ii. wait3():** Suspends execution of current process until a child has exited or until signal is delivered.

```
pid_t wait3(int *status, int options, struct rusage *rusage);
```
- iii. wait4():** As same as wait3() but includes

```
pid_t pid value.pid_t wait3(pid_t pid, int *status, int options, struct
rusage *rusage);
```

4) `exec( )` : `exec()` family of functions or sys calls replaces current process image with new process image.

There are functions like `execl`, `execlp`, `execle`, `execv`, `execvp` and `execvpe` are used to execute a file.

These functions are combinations of array of pointers to null terminated strings that represent the argument list, this will have path variable with some environment variable combinations.

5) `exit()`: This function is used for normal process termination. The status of the process is captured for future reference. There are other similar functions `exit(3)` and `_exit()`, which are used based on the exiting process that one is interested to use or capture.

UNIT – II

REAL TIME OPERATING SYSTEMS

BRIEF HISTORY OF OS

An Operating system (OS) is a software which acts as an interface between the end user and computer hardware. Every computer must have at least one OS to run other programs. An application like Chrome, MS Word, Games, etc needs some environment in which it will run and perform its task. The OS helps you to communicate with the computer without knowing how to speak the computer's language. It is not possible for the user to use any computer or mobile device without having an operating system.

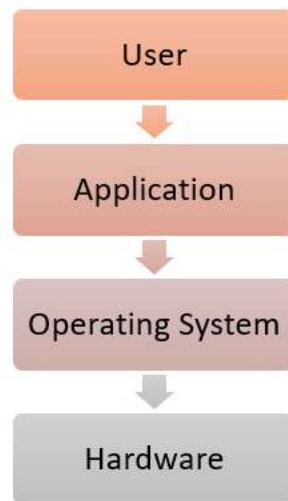
History Of OS:

- ⇒ Operating systems were first developed in the late 1950s to manage tape storage.
- ⇒ The General Motors Research Lab implemented the first OS in the early 1950s for their IBM 701
- ⇒ In the mid-1960s, operating systems started to use disks.
- ⇒ In the late 1960s, the first version of the Unix OS was developed.
- ⇒ The first OS built by Microsoft was DOS. It was built in 1981 by purchasing the 86-DOS software from a Seattle company.
- ⇒ The present-day popular OS Windows first came to existence in 1985 when a GUI was created and paired with MS-DOS.

Features of Operating System: Here is a list commonly found important features of an Operating System:

- 1) Protected and supervisor mode.
- 2) Allows disk access and file systems Device drivers Networking Security.
- 3) Program Execution.
- 4) Memory management Virtual Memory Multitasking.
- 5) Handling I/O operations.
- 6) Manipulation of the file system.
- 7) Error Detection and handling.
- 8) Resource allocation.

9) Information and Resource Protection.



DEFINING REAL-TIME OPERATING SYSTEM (RTOS)

A real-time operating system (RTOS) is an operating system (OS) intended to serve real-time applications that process data as it comes in, typically without buffer delays. Processing time requirements (including any OS delay) are measured in tenths of seconds or shorter increments of time. A real-time system is a time bound system which has well defined fixed time constraints. Processing must be done within the defined constraints or the system will fail. They either are event driven or time sharing. Event driven systems switch between tasks based on their priorities while time sharing systems switch the task based on clock interrupts. Most RTOSs use a pre-emptive scheduling algorithm.

A key characteristic of an RTOS is the level of its consistency concerning the amount of time it takes to accept and complete an application's task; the variability is 'jitter'. A 'hard' real-time operating system has less jitter than a 'soft' real-time operating system. The chief design goal is not high throughput, but rather a guarantee of a soft or hard performance category. An RTOS that can usually or generally meet a deadline is a soft real-time OS, but if it can meet a deadline deterministically it is a hard real-time OS.

An RTOS has an advanced algorithm for scheduling. Scheduler flexibility enables a wider, computer-system orchestration of process priorities, but a real-time OS is more frequently dedicated to a narrow set of applications. Key

factors in a real-time OS are minimal interrupt latency and minimal thread switching latency; a real-time OS is valued more for how quickly or how predictably it can respond than for the amount of work it can perform in a given period of time.

THE SCHEDULER

Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run. Schedulers are of three types:

- 1) Long-Term Scheduler.
- 2) Short-Term Scheduler.
- 3) Medium-Term Scheduler.

1) Long Term Scheduler: It is also called a job scheduler. A long-term scheduler determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution. Process loads into the memory for CPU scheduling.

The primary objective of the job scheduler is to provide a balanced mix of jobs, such as I/O bound and processor bound. It also controls the degree of multiprogramming. If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system.

On some systems, the long-term scheduler may not be available or minimal. Time-sharing operating systems have no long term scheduler. When a process changes the state from new to ready, then there is use of long-term scheduler.

2) Short Term Scheduler: It is also called as CPU scheduler. Its main objective is to increase system performance in accordance with the chosen set of criteria. It is the change of ready state to running state of the process. CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.

Short-term schedulers, also known as dispatchers, make the decision of which process to execute next. Short-term schedulers are faster than long-term schedulers.

3) Medium Term Scheduler: Medium-term scheduling is a part of swapping. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium-term scheduler is in-charge of handling the swapped out-processes.

A running process may become suspended if it makes an I/O request. Suspended processes cannot make any progress towards completion. In this condition, to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage. This process is called swapping, and the process is said to be swapped out or rolled out. Swapping may be necessary to improve the process mix.

Comparison among Scheduler:

SNo.	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a job scheduler.	It is a CPU scheduler.	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler.	Speed is fastest among other two.	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming.	It provides lesser control over degree of multiprogramming.	It reduces the degree of multiprogramming.
4	It is almost absent or minimal in time sharing system.	It is also minimal in time sharing system.	It is a part of Time sharing systems.
5	It selects processes from pool and loads them into memory for execution.	It selects those processes which are ready to execute.	It can re-introduce the process into memory and execution can be continued.

OBJECTS

Special constructs like tasks, semaphores or message queues.

Kernel Objects: Kernel objects are special constructs that are the building blocks for application development for real-time embedded systems. The

various kernel objects are Tasks, Task Scheduler, Interrupt Service Routines, Semaphores, Mutexes, Mailboxes, Message Queues, Pipes, Timers etc.

SERVICES

An Operating System provides services to both the users and to the programs. It provides programs an environment to execute. It provides users the services to execute the programs in a convenient manner. Following are a few common services provided by an operating system:

- 1) Program execution.
- 2) I/O operations.
- 3) File System manipulation.
- 4) Communication.
- 5) Error Detection.
- 6) Resource Allocation.
- 7) Protection.

1) Program execution: Operating systems handle many kinds of activities from user programs to system programs like printer spooler, name servers, file server, etc. Each of these activities is encapsulated as a process.

A process includes the complete execution context (code to execute, data to manipulate, registers, OS resources in use). Following are the major activities of an operating system with respect to program management:

- i. Loads a program into memory.
- ii. Executes the program.
- iii. Handles program's execution.
- iv. Provides a mechanism for process synchronization.
- v. Provides a mechanism for process communication.
- vi. Provides a mechanism for deadlock handling.

2) I/O Operation: An I/O subsystem comprises of I/O devices and their corresponding driver software. Drivers hide the peculiarities of specific hardware devices from the users.

An Operating System manages the communication between user and device drivers. I/O operation means read or write operation with any file or any

specific I/O device. Operating system provides the access to the required I/O device when required.

3) File system manipulation: A file represents a collection of related information. Computers can store files on the disk (secondary storage), for long-term storage purpose. Examples of storage media include magnetic tape, magnetic disk and optical disk drives like CD, DVD. Each of these media has its own properties like speed, capacity, data transfer rate and data access methods.

A file system is normally organized into directories for easy navigation and usage. These directories may contain files and other directions. Following are the major activities of an operating system with respect to file management:

- i. Program needs to read a file or write a file.
- ii. The operating system gives the permission to the program for operation on file.
- iii. Permission varies from read-only, read-write, denied and so on.
- iv. Operating System provides an interface to the user to create/delete files.
- v. Operating System provides an interface to the user to create/delete directories.
- vi. Operating System provides an interface to create the backup of file system.

4) Communication: In case of distributed systems which are a collection of processors that do not share memory, peripheral devices, or a clock, the operating system manages communications between all the processes. Multiple processes communicate with one another through communication lines in the network.

The OS handles routing and connection strategies, and the problems of contention and security. Following are the major activities of an operating system with respect to communication:

- i. Two processes often require data to be transferred between them.

- ii. Both the processes can be on one computer or on different computers, but are connected through a computer network.
- iii. Communication may be implemented by two methods, either by Shared Memory or by Message Passing.

5) Error handling: Errors can occur anytime and anywhere. An error may occur in CPU, in I/O devices or in the memory hardware. Following are the major activities of an operating system with respect to error handling:

- i. The OS constantly checks for possible errors.
- ii. The OS takes an appropriate action to ensure correct and consistent computing.

6) Resource Management: In case of multi-user or multi-tasking environment, resources such as main memory, CPU cycles and files storage are to be allocated to each user or job. Following are the major activities of an operating system with respect to resource management:

- i. The OS manages all kinds of resources using schedulers.
- ii. CPU scheduling algorithms are used for better utilization of CPU.

7) Protection: Considering a computer system having multiple users and concurrent execution of multiple processes, the various processes must be protected from each other's activities.

Protection refers to a mechanism or a way to control the access of programs, processes, or users to the resources defined by a computer system. Following are the major activities of an operating system with respect to protection:

- i. The OS ensures that all access to system resources is controlled.
- ii. The OS ensures that external I/O devices are protected from invalid access attempts.
- iii. The OS provides authentication features for each user by means of passwords.

CHARACTERISTICS OF RTOS

Characteristics of real-time systems and their issues related to designing both soft and hard real-time operating systems are given below. The following characteristics are typical of many real-time systems:

- 1) Simple and Single purpose.
- 2) Small size.
- 3) Inexpensively mass-produced.
- 4) Specific timing requirements.
- 5) Reliability.
- 6) Performance.

1) Simple and Single Purpose: A Real-Time Operating System is basically designed for a single purpose only. It doesn't serve the various applications within a single platform. E.g: Washing machine is an embedded real-time machine which is used for washing the clothes only. Same like a microwave oven which is used just for cooking the food. RTOS don't have use for multiple purposes so it's a simple in design.

2) Small Size: Many real-time systems exist in environments where physical space is limited. E.g: Space required for wristwatch or microwave is very less as compared to a Computer system. So we can say that the real-time system doesn't require both CPU power and memory. Whereas most PC and server systems have 32 or 64-bit processors, many real-time systems run on 8 or 16-bit processors. Similarly, a desktop PC might have several gigabytes of physical memory, whereas a real-time system might have less than a megabyte.

3) Inexpensively mass-produced: The Real-time systems are found mostly in home appliances and consumer devices. Cost of such devices like digital cameras, microwave ovens, and thermostats is really affordable to the users. So these are mass-produced in very cost-conscious environments. Thus, the microprocessors for real-time systems must also be inexpensively mass-produced.

4) Specific timing requirements: Real-time operating systems should consider timing requirements by using scheduling algorithms that give real-time processes the highest scheduling priorities. Scheduler decides the priority of tasks and priority of tasks does not degrade over time. Timing requirements can be addressed by minimizing the response time of interrupts.

A critical real-time system requires sufficient time for processing external events. This is known as the response time and it lies within a predetermined value in all possible situations. Basically, a real-time system is characterized by correctness that involves both the correctness and timeliness of the logical output.

5) Reliability: Some systems might be operated for a long time without user intervention. The real-time system provides the reliability that it surely produces a result in a given time and chooses the most profitable activity for the current operation.

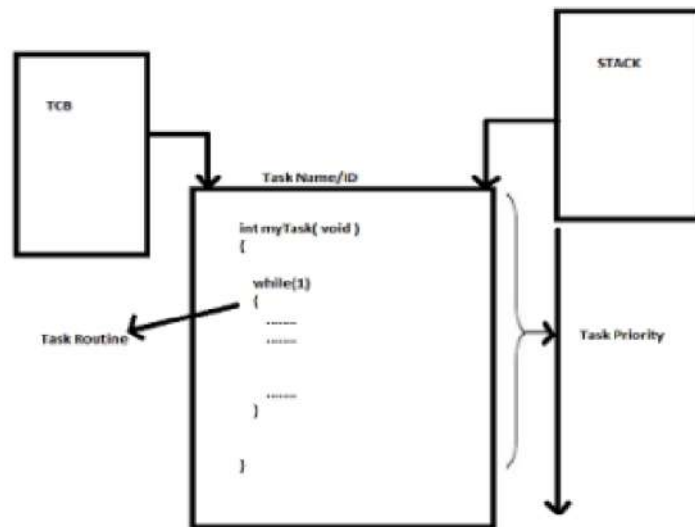
6) Performance: Performance is an important aspect that must be considered when choosing an RTOS. Real-time operating systems are different and perform differently. The key aspect for an RTOS is that its determinism guarantees that request and responses of data happen within a set period of time no matter what else is happening in the PC system.

When determining the best RTOS, ask questions such as whether the system is showing any jitters within your tolerance range and thereby providing the determinism that you need. RTOS performance should be determined by a system's dependable in executing calls within a specified period, regardless of anything else happening on the system.

DEFINING A TASK

A task is a basic unit of execution that can be scheduled by an RTOS to use the system resources. In other words, a task is an independent thread of execution that compete with other concurrent tasks for process execution time. A task is define by its distinct set of parameter and supporting data structure. Specially upon creation, each task has an associated name, a

unique id, a priority, a Task Control Block (TCB). a stack, a task routine. Together these components make up what is known as task object.



In RTOS implementation of a design, the program is divided into different independent functions what we call as a task. These functions are not called anywhere in the program, they are just created. Each task runs continuously i.e., in infinite loop. The kernel is responsible for the management of all the tasks.

TASKS STATES AND SCHEDULING

Different states of a task:

In the FreeRTOS a task can be in either of four different states viz., Running, Ready, Blocked and Suspended. Let's see each state one by one.

- 1) Running:** The task which is executing currently is said to be in running state. It owns the CPU.
- 2) Ready:** The task which is neither suspended nor blocked but still not executing will be ready state. It's not in running state because either a high priority or equal priority task is executing.
- 3) Blocked:** A task will go in blocked state whenever it's waiting for an event to happen. The event can be completing a delay period or availability of a resource. The blocked tasks are not available for scheduling.

4) Suspended: When `vTaskSuspend()` is called, the task goes in suspended state. To resume it, `xTaskResume()` is called. The suspended tasks are also not available for scheduling.

It seems that the tasks in both Blocked and Suspended are in waiting state. But the difference is, a suspended task can't come in Ready state unless `xTaskResume()` is called. Time out period can be specified only for Blocked tasks but not for suspended tasks.

Task scheduling with a real time operating system:

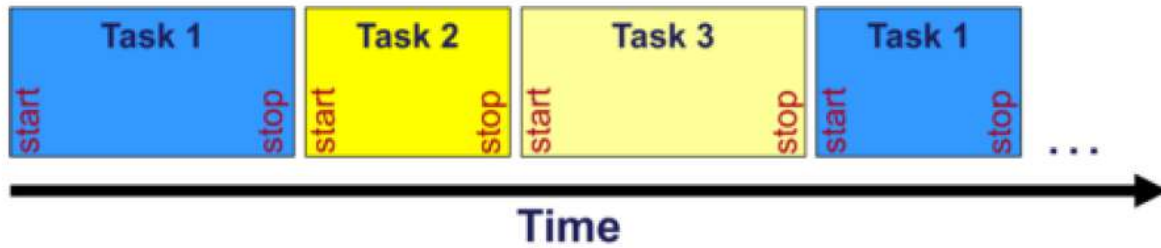
From time to time, I like to poke around at some RTOS feature or functionality. Today I want to ponder the real core of any operating system: the scheduler. Every embedded system is different and it is, therefore, unsurprising that there are a number of different types of scheduler that might be employed. Different RTOS products offer different options, but I want to try to cover the spectrum of possibilities ...

As any software engineer knows, a conventional CPU can only execute one program at a time. The apparent simultaneous execution of multiple threads is achieved by control of the CPU being rapidly swapped between a number of code threads. This swapping process is managed by the scheduler.

A scheduler need not be particularly complex. I have written a few in the course of my career and have never quite forgotten the first time I could clearly observe some code of mine unambiguously multitasking; it seemed somewhat magical, even though I knew exactly what was going on [as I had written every instruction of code involved].

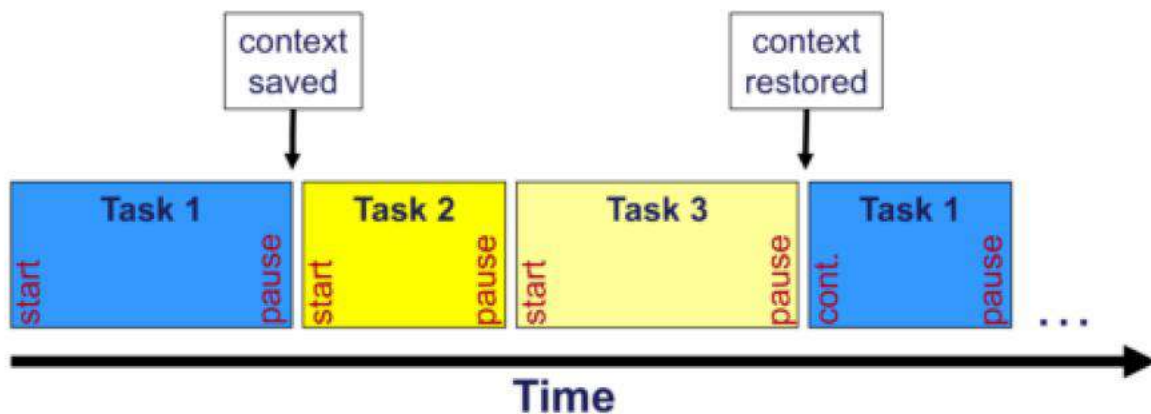
Although it is a slight simplification, I can identify five broad types of scheduler:

1) Run to completion [RTC]: An RTC scheduler is very simple. Indeed, I have previously [and only slight inaccurately] referred to one as a “one line RTOS”. The idea is that one task runs until it has completed its work, then terminates. Then the next task runs similarly. And so forth until all the tasks have run, when the sequence starts again.



The simplicity of this scheme is offset by the drawback that each task's allocation of time is totally affected by all the others. The system will not be very deterministic. But, for some applications, this is quite satisfactory. An added level of sophistication might be support for task suspend, which means that one or more tasks may be excluded from the execution sequence until they are required again.

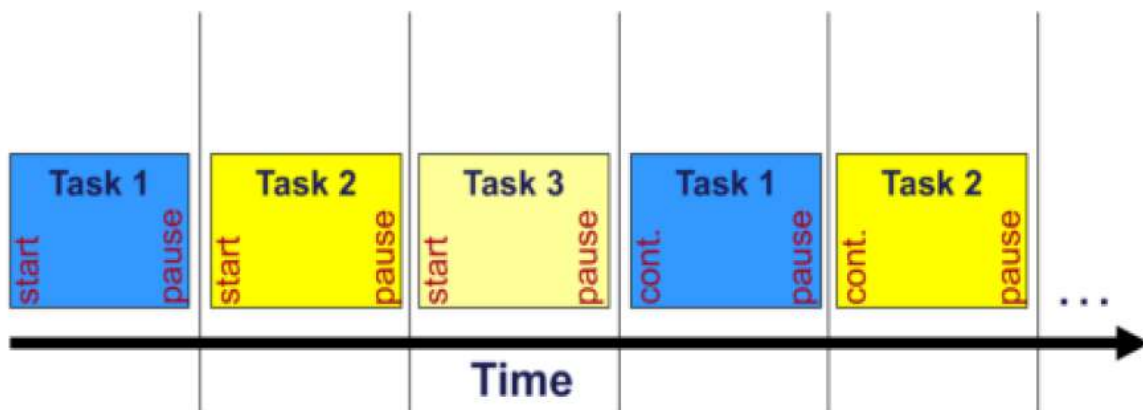
2) Round robin [RR]: An RR scheduler is the next level of complexity. Tasks are run in sequence in just the same way [with task suspend being a possibility], except that a task does not need to complete its work, it just relinquishes the CPU when convenient to do so. When it is scheduled again, it continues from where it left off.



The greater flexibility of an RR scheduler comes at the cost of complexity. When a task relinquishes the CPU, its context [basically machine register values] needs to be saved so that it can be restored next time the task is scheduled. This process is required for all the other scheduler varieties that I will discuss.

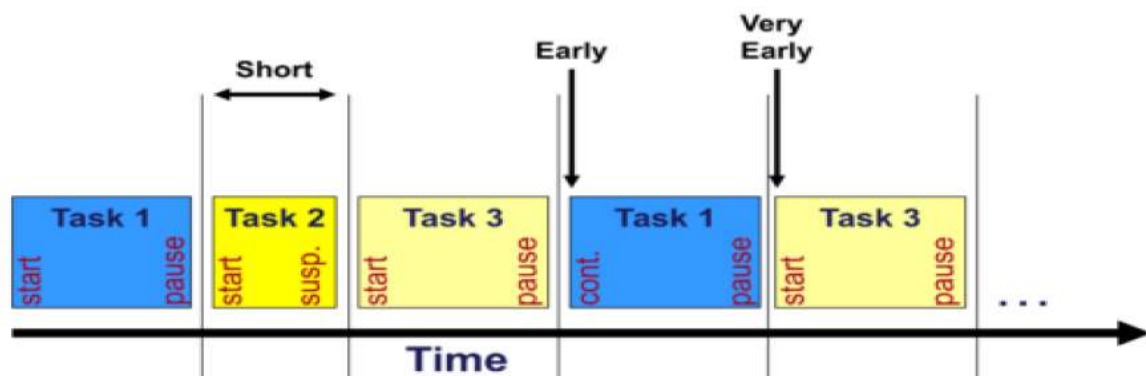
As with RTC, an RR scheduler still relies of each task behaving well and not hanging on to the processor for too long. Both RTC and RR are “cooperative multitasking”.

3) Time slice [TS]: A TS scheduler is a straightforward example of “preemptive multitasking”. The idea is to divide time into “slots”, each of which might be, say, 1mS. Each task gets to run in a slot. At the end of its allocated time, it is interrupted and the next task run.

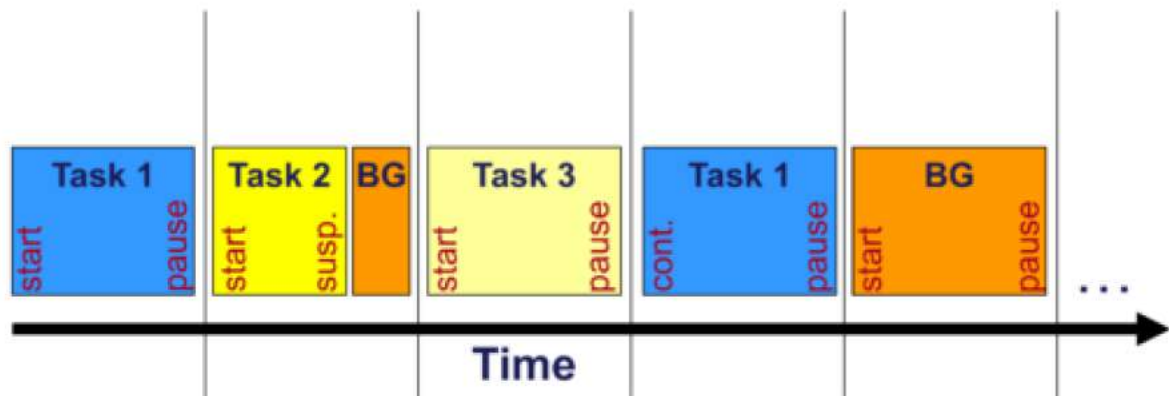


The scheduling is not now dependent on tasks being “good citizens”, as time utilization is managed fairly. A system built with a TS scheduler may be fully deterministic [i.e., predictable] – it is truly real time.

4) Time slice with background task [TSBG]: Although a TS scheduler is neat and tidy, there is a problem. If a task finds that it has no work to do, its only option is to loop – burning CPU time – until it can do something useful. This means that it might waste a significant proportion of its slot and an indefinite number of further slots. Clearly, the task might suspend itself [to be woken again when it is needed], but this messes up the timing of the other tasks.

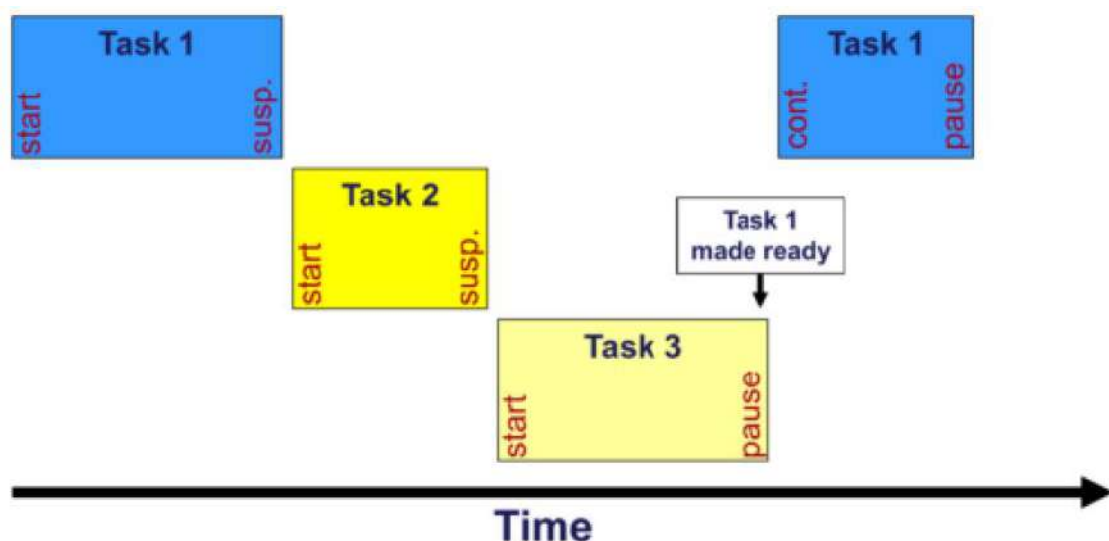


This is unfortunate, as the determinism of the system is compromised. A solution is to enhance the scheduler so that, if a task suspends itself, the remainder of its slot is taken up by a “background task”; this task would also use the full slots of any suspended tasks. This restores the timing integrity.



What the background task actually does depends on the application, but broadly it must be non-time-critical code – like self-testing. There is, of course, the possibility that the background task will never get scheduled. Also, this special task cannot be suspended.

5) Priority [PRI]: A common, more sophisticated scheduling scheme is PRI, which is used in many [most] commercial RTOS products. The idea is that each task has a priority and is either “ready” [to run] or “suspended”. The scheduler runs the task with the highest priority that is “ready”. When that task suspends, it runs the one with the next highest priority. If an event occurs, which may have readied a higher priority task, the scheduler is run.



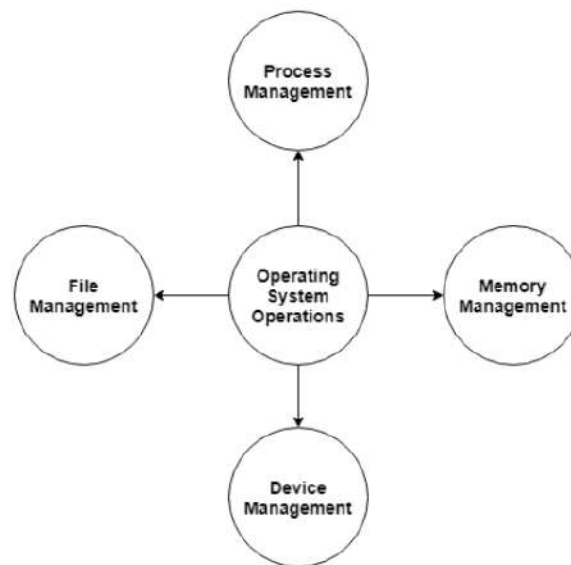
Although more complex, a PRI scheduler give most flexibility for many applications.

Commercial RTOS products, like our own Nucleus RTOS, tend to use a priority scheduling scheme, but allow multiple tasks at each priority level. A time slice mechanism is then employed to allocate CPU time between multiple “ready” tasks of the same priority.

TASK OPERATIONS

An operating system is a construct that allows the user application programs to interact with the system hardware. Operating system by itself does not provide any function but it provides an atmosphere in which different applications and programs can do useful work.

The major operations of the operating system are process management, memory management, device management and file management. These are given in detail as follows:



1) Process Management: The operating system is responsible for managing the processes i.e., assigning the processor to a process at a time. This is known as process scheduling. The different algorithms used for process scheduling are FCFS (first come first served), SJF (shortest job first), priority scheduling, round robin scheduling etc.

There are many scheduling queues that are used to handle processes in process management. When the processes enter the system, they are put into the job queue. The processes that are ready to execute in the main memory are kept in the ready queue. The processes that are waiting for the I/O device are kept in the device queue.

2) Memory Management: Memory management plays an important part in operating system. It deals with memory and the moving of processes from disk to primary memory for execution and back again.

The activities performed by the operating system for memory management are: The operating system assigns memory to the processes as required. This can be done using best fit, first fit and worst fit algorithms. All the memory is tracked by the operating system i.e., it's nodes what memory parts are in use by the processes and which are empty.

The operating system deallocated memory from processes as required. This may happen when a process has been terminated or if it no longer needs the memory.

3) Device Management: There are many I/O devices handled by the operating system such as mouse, keyboard, disk drive etc. There are different device drivers that can be connected to the operating system to handle a specific device. The device controller is an interface between the device and the device driver. The user applications can access all the I/O devices using the device drivers, which are device specific codes.

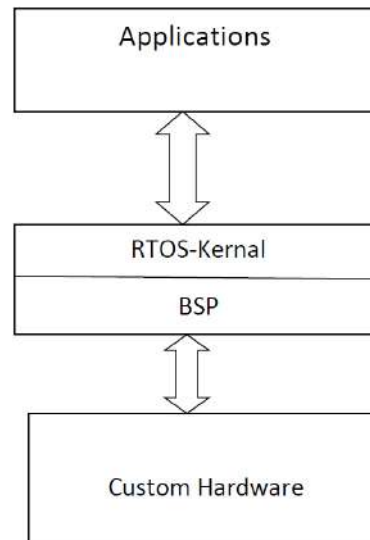
4) File Management: Files are used to provide a uniform view of data storage by the operating system. All the files are mapped onto physical devices that are usually non volatile so data is safe in the case of system failure.

The files can be accessed by the system in two ways i.e., sequential access and direct access:

- i. Sequential Access:** The information in a file is processed in order using sequential access. The files records are accessed one after another. Most of the file systems such as editors, compilers etc. use sequential access.

- ii. Direct Access:** In direct access or relative access, the files can be accessed in random for read and write operations. The direct access model is based on the disk model of a file, since it allows random accesses.

TASK STRUCTURE



Types of RTOS: There are three different types of RTOS which are following:

- 1) Soft Real-Time Operating System:** The soft real-time operating system has certain deadlines, may be missed and they will take the action at a time $t = 0+$. The soft real-time operating system is a type of OS and it does not contain constrained to extreme rules. The critical time of this operating system is delayed to some extent. The examples of this operating system are the digital camera, mobile phones and online data etc.
- 2) Hard Real-Time Operating System:** This is also a type of OS and it is predicted by a deadline. The predicted deadlines will react at a time $t = 0$. Some examples of this operating system are air bag control in cars, anti-lock brake, and engine control system etc.
- 3) Firm Real-Time Operating System:** In the firm real-time, an operating system has certain time constraints, they are not strict and it may cause undesired effects. Examples of this operating system are a visual inspection in industrial automation.

SYNCHRONIZATION

Synchronization is classified into two categories: resource synchronization and activity synchronization. Resource synchronization determines whether access to a shared resource is safe, and, if not, when it will be safe. Activity synchronization determines whether the execution of a multithreaded program has reached a certain state and, if it hasn't, how to wait for and be notified when this state is reached.

1) Resource Synchronization: Access by multiple tasks must be synchronized to maintain the integrity of a shared resource. This process is called resource synchronization, a term closely associated with critical sections and mutual exclusions.

Mutual exclusion is a provision by which only one task at a time can access a shared resource. A critical section is the section of code from which the shared resource is accessed.

As an example, consider two tasks trying to access shared memory. One task (the sensor task) periodically receives data from a sensor and writes the data to shared memory. Meanwhile, a second task (the display task) periodically reads from shared memory and sends the data to a display. The common design pattern of using shared memory is illustrated in Figure 15.1.

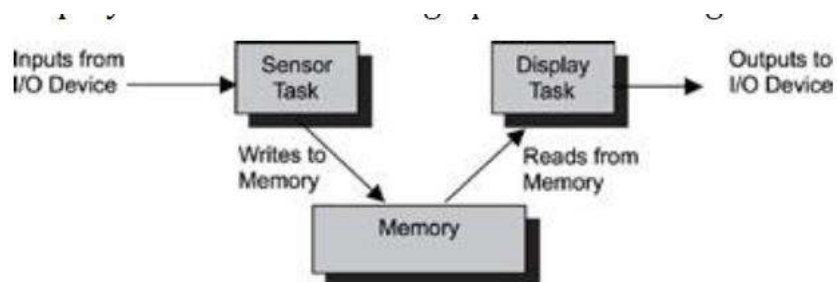


Figure 15.1: Multiple tasks accessing shared memory.

Problems arise if access to the shared memory is not exclusive, and multiple tasks can simultaneously access it. For example, if the sensor task has not completed writing data to the shared memory area before the display task tries to display the data, the display would contain a mixture of data extracted at different times, leading to erroneous data interpretation.

The section of code in the sensor task that writes input data to the shared memory is a critical section of the sensor task. The section of code in the display task that reads data from the shared memory is a critical section of the display task. These two critical sections are called competing critical sections because they access the same shared resource.

A mutual exclusion algorithm ensures that one task's execution of a critical section is not interrupted by the competing critical sections of other concurrently executing tasks.

One way to synchronize access to shared resources is to use a client-server model, in which a central entity called a resource server is responsible for synchronization. Access requests are made to the resource server, which must grant permission to the requestor before the requestor can access the shared resource. The resource server determines the eligibility of the requestor based on pre-assigned rules or run-time heuristics.

While this model simplifies resource synchronization, the resource server is a bottleneck. Synchronization primitives, such as semaphores and mutexes, and other methods introduced in a later section of this chapter, allow developers to implement complex mutual exclusion algorithms. These algorithms in turn allow dynamic coordination among competing tasks without intervention from a third party.

2) Activity Synchronization: In general, a task must synchronize its activity with other tasks to execute a multithreaded program properly. Activity synchronization is also called condition synchronization or sequence control. Activity synchronization ensures that the correct execution order among cooperating tasks is used. Activity synchronization can be either synchronous or asynchronous.

One representative of activity synchronization methods is barrier synchronization. For example, in embedded control systems, a complex computation can be divided and distributed among multiple tasks. Some parts of this complex computation are I/O bound, other parts are CPU intensive, and still others are mainly floating-point operations that rely

heavily on specialized floating-point coprocessor hardware. These partial results must be collected from the various tasks for the final calculation. The result determines what other partial computations each task is to perform next.

The point at which the partial results are collected and the duration of the final computation is a barrier. One task can finish its partial computation before other tasks complete theirs, but this task must wait for all other tasks to complete their computations before the task can continue.

Barrier synchronization comprises three actions:

- i. a task posts its arrival at the barrier,
- ii. the task waits for other participating tasks to reach the barrier, and
- iii. the task receives notification to proceed beyond the barrier.

As shown in Figure 15.2, a group of five tasks participates in barrier synchronization. Tasks in the group complete their partial execution and reach the barrier at various times; however, each task in the group must wait at the barrier until all other tasks have reached the barrier. The last task to reach the barrier (in this example, task T5) broadcasts a notification to the other tasks. All tasks cross the barrier at the same time (conceptually in a uniprocessor environment due to task scheduling. We say 'conceptually' because in a uniprocessor environment, only one task can execute at any given time. Even though all five tasks have crossed the barrier and may continue execution, the task with the highest priority will execute next.

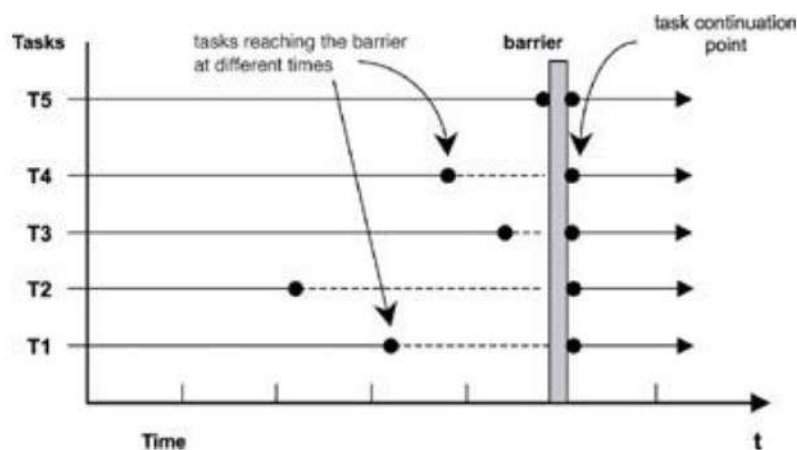


Figure 15.2: Visualization of barrier synchronization.

Another representative of activity synchronization mechanisms is rendezvous synchronization, which, as its name implies, is an execution point where two tasks meet. The main difference between the barrier and the rendezvous is that the barrier allows activity synchronization among two or more tasks, while rendezvous synchronization is between two tasks.

In rendezvous synchronization, a synchronization and communication point called an entry is constructed as a function call. One task defines its entry and makes it public. Any task with knowledge of this entry can call it as an ordinary function call. The task that defines the entry accepts the call, executes it, and returns the results to the caller. The issuer of the entry call establishes a rendezvous with the task that defined the entry.

Rendezvous synchronization is similar to synchronization using event-registers, which Chapter 8 introduces, in that both are synchronous. The issuer of the entry call is blocked if that call is not yet accepted; similarly, the task that accepts an entry call is blocked when no other task has issued the entry call. Rendezvous differs from event-register in that bidirectional data movement (input parameters and output results) is possible.

A derivative form of rendezvous synchronization, called simple rendezvous in this book, uses kernel primitives, such as semaphores or message queues, instead of the entry call to achieve synchronization. Two tasks can implement a simple rendezvous without data passing by using two binary semaphores, as shown in Figure 15.3.

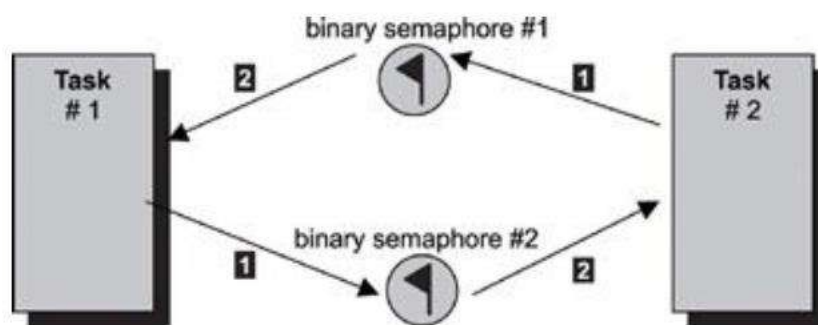


Figure 15.3: Simple rendezvous without data passing.

Both binary semaphores are initialized to 0. When task #1 reaches the rendezvous, it gives semaphore #2, and then it gets on semaphore #1. When

task #2 reaches the rendezvous, it gives semaphore #1, and then it gets on semaphore #2. Task #1 has to wait on semaphore #1 before task #2 arrives, and vice versa, thus achieving rendezvous synchronization.

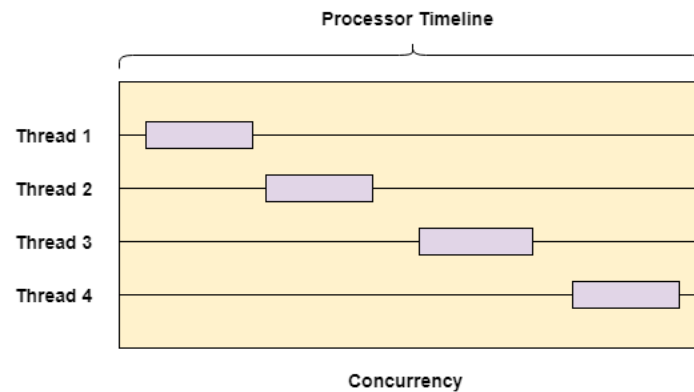
COMMUNICATION

Since only one task can be running at one time (remember the book analogy), there must be mechanisms for tasks to communicate with one another. ñ A task is reading data from a sensor at 15 hz. It stores 1024 bytes of data and then needs to signal a processing task to take and process the data so it has room to write more. ñ A task is determining the state of a system- i.e. Normal Mode, Urgent Mode, Sleeping, Disabled. It needs to inform all other tasks in the system of a change in status. ñ A user is communicating to another user across a network. The network receive task has to deliver messages to the terminal program, and the terminal program has to deliver messages to the network transmit task.

Regular operating systems have many options for passing messages between processes, but most involve significant overhead and aren't deterministic. ñ Pipes, message queues, semaphores, Remote Procedure Calls, Sockets, Datagrams, etc. ï In a RTOS, tasks generally have direct access to a common memory space, and the fastest way to share data is by sharing memory. ñ In ordinary OSs, tasks are usually prevented from accessing another task's memory, and for good reason.

CONCURRENCY

Concurrency means that multiple processes or threads are making progress concurrently. While only one thread is executed at a time by the CPU, these threads can be switched in and out as required. This means that no thread is actually completed totally before another is scheduled. So all the threads are executing concurrently. An image that demonstrates concurrency is as follows:



In the above diagram, all the four threads are running concurrently. However, only one of them can be scheduled on a processor at a time.

DEFINING SEMAPHORES: OPERATIONS AND USE

Consider a situation where there are two person who wants to share a bike. At one time only one person can use the bike. The one who has the bike key will get the chance to use it. And when this person gives the key to 2nd person, then only 2nd person can use the bike.

Semaphore is just like this Key and the bike is the shared resource. Whenever a task wants access to the shared resource, it must acquire the semaphore first. The task should release the semaphore after it is done with the shared resource. Till this time all other tasks have to wait if they need access to shared resource as semaphore is not available. Even if the task trying to acquire the semaphore is of higher priority than the task acquiring the semaphore, it will be in wait state until semaphore is released by the lower priority task.

Use of Semaphore:

- 1) Managing Shared Resource.
- 2) **Task Synchronization:** Apart from managing shared resource, task synchronization can also be performed with the help of a semaphore. In this case semaphore will be like a flag not key.
- 3) **Unilateral Rendezvous:** This is one way synchronization which uses a semaphore as a flag to signal another task.
- 4) **Bilateral Rendezvous:** This is two way synchronization performed using two semaphores. A bilateral rendezvous is similar to a unilateral

rendezvous, except both tasks must synchronize with one another before proceeding.

Operations on Semaphore: Basically, there are 3 operations related to the semaphore:

- 1) Create.
- 2) Acquire.
- 3) Release.

Semaphores are integer variables that are used to solve the critical section problem by using two atomic operations, wait and signal that are used for process synchronization.

The definitions of wait and signal are as follows:

Wait

The wait operation decrements the value of its argument S, if it is positive. If S is negative or zero, then no operation is performed.

```
wait(S)
{
while (S<=0);

S--;
}
```

Signal

The signal operation increments the value of its argument S.

```
signal(S)
{
S++;
}
```

Types of Semaphores: There are two main types of semaphores i.e. counting semaphores and binary semaphores. Details about these are given as follows:

- 1) Counting Semaphores:** These are integer value semaphores and have an unrestricted value domain. These semaphores are used to coordinate the resource access, where the semaphore count is the

number of available resources. If the resources are added, semaphore count automatically incremented and if the resources are removed, the count is decremented.

2) Binary Semaphores: The binary semaphores are like counting semaphores but their value is restricted to 0 and 1. The wait operation only works when the semaphore is 1 and the signal operation succeeds when semaphore is 0. It is sometimes easier to implement binary semaphores than counting semaphores.

Advantages of Semaphores: Some of the advantages of semaphores are as follows:

- 1) Semaphores allow only one process into the critical section. They follow the mutual exclusion principle strictly and are much more efficient than some other methods of synchronization.
- 2) There is no resource wastage because of busy waiting in semaphores as processor time is not wasted unnecessarily to check if a condition is fulfilled to allow a process to access the critical section.
- 3) Semaphores are implemented in the machine independent code of the microkernel. So they are machine independent.

Disadvantages of Semaphores: Some of the disadvantages of semaphores are as follows:

- 1) Semaphores are complicated so the wait and signal operations must be implemented in the correct order to prevent deadlocks.
- 2) Semaphores are impractical for large scale use as their use leads to loss of modularity. This happens because the wait and signal operations prevent the creation of a structured layout for the system.
- 3) Semaphores may lead to a priority inversion where low priority processes may access the critical section first and high priority processes later.

DEFINING MESSAGE QUEUE

A message queue is a buffer-like object through which tasks and ISRs send and receive messages to communicate and synchornize with data. A message queue is like a pipeline. It temporarily holds messages from a sender until the intended receiver is ready to read them. This temporary buffering decouples a sending and receiving task; that is, it frees the tasks from having to send and receive messages simultaneously.

A message queue has several associated components that the kernel uses to manage the queue. When a message queue is first created, it is assigned an associated queue control block (QCB), a message queue name, a unique ID, memory buffers, a queue length, a maximum message length, and one or more task-waiting lists, as illustrated in Figure 7.1.

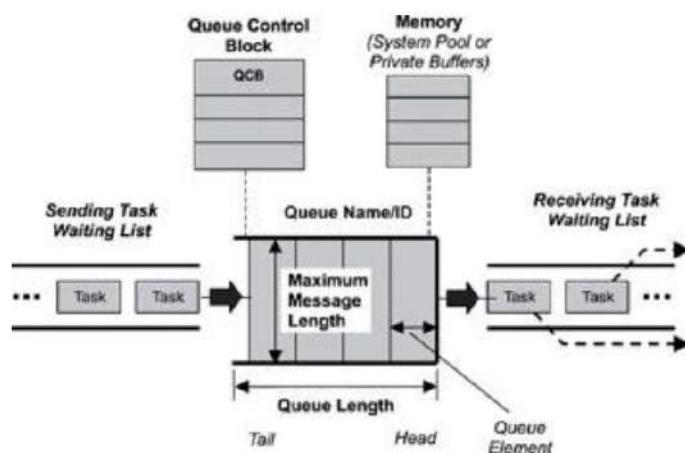


Figure 7.1: A message queue, its associated parameters, and supporting data structures.

It is the kernel's job to assign a unique ID to a message queue and to create its QCB and task-waiting list. The kernel also takes developer-supplied parameters—such as the length of the queue and the maximum message length—to determine how much memory is required for the message queue. After the kernel has this information, it allocates memory for the message queue from either a pool of system memory or some private memory space.

The message queue itself consists of a number of elements, each of which can hold a single message. The elements holding the first and last messages are called the head and tail respectively. Some elements of the queue may be empty (not containing a message). The total number of elements (empty or

not) in the queue is the total length of the queue. The developer specified the queue length when the queue was created.

As Figure 7.1 shows, a message queue has two associated task-waiting lists. The receiving task-waiting list consists of tasks that wait on the queue when it is empty. The sending list consists of tasks that wait on the queue when it is full.

MESSAGE QUEUE STATES, CONTENT

As with other kernel objects, message queues follow the logic of a simple FSM, as shown in Figure 7.2. When a message queue is first created, the FSM is in the empty state. If a task attempts to receive messages from this message queue while the queue is empty, the task blocks and, if it chooses to, is held on the message queue's task-waiting list, in either a FIFO or priority-based order.

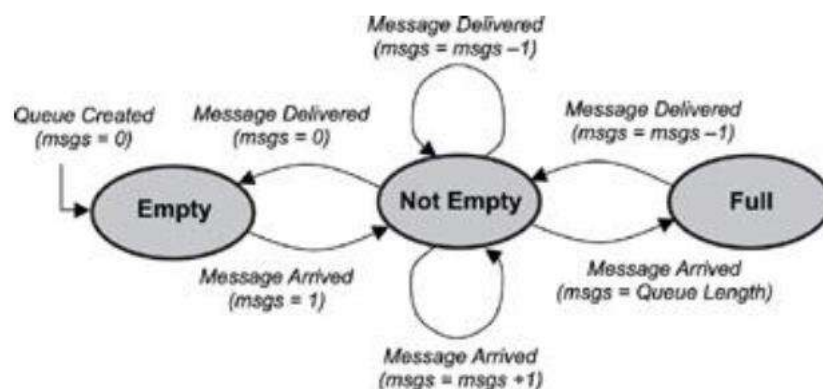


Figure 7.2: The state diagram for a message queue.

In this scenario, if another task sends a message to the message queue, the message is delivered directly to the blocked task. The blocked task is then removed from the task-waiting list and moved to either the ready or the running state. The message queue in this case remains empty because it has successfully delivered the message.

If another message is sent to the same message queue and no tasks are waiting in the message queue's task-waiting list, the message queue's state becomes not empty.

As additional messages arrive at the queue, the queue eventually fills up until it has exhausted its free space. At this point, the number of messages in the queue is equal to the queue's length, and the message queue's state becomes full. While a message queue is in this state, any task sending messages to it will not be successful unless some other task first requests a message from that queue, thus freeing a queue element.

In some kernel implementations when a task attempts to send a message to a full message queue, the sending function returns an error code to that task. Other kernel implementations allow such a task to block, moving the blocked task into the sending task-waiting list, which is separate from the receiving task-waiting list.

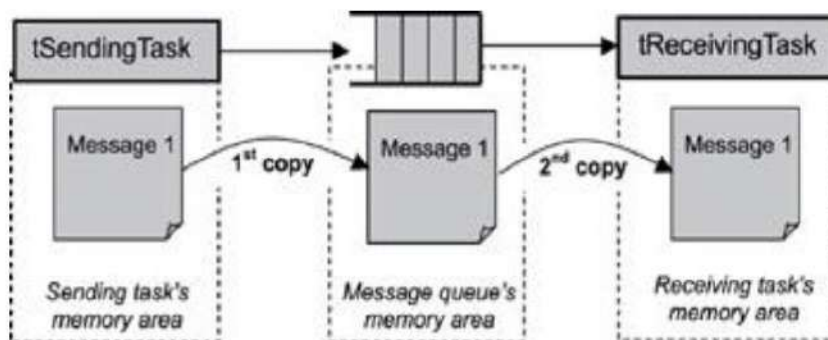
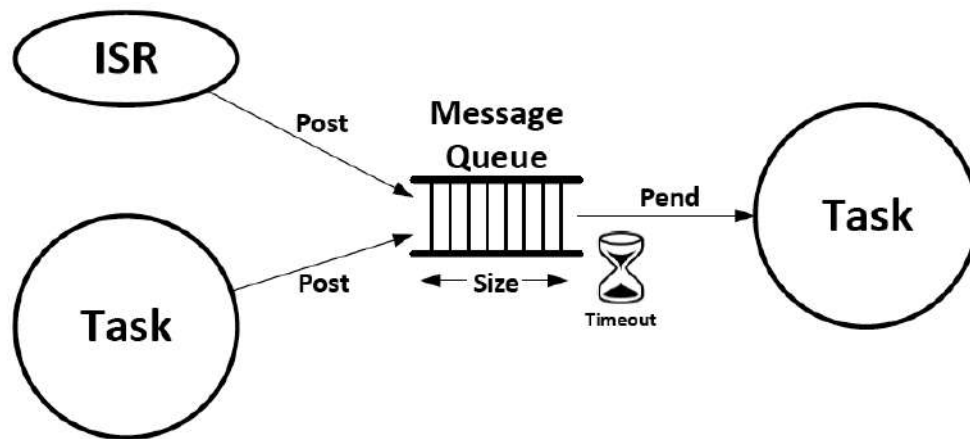


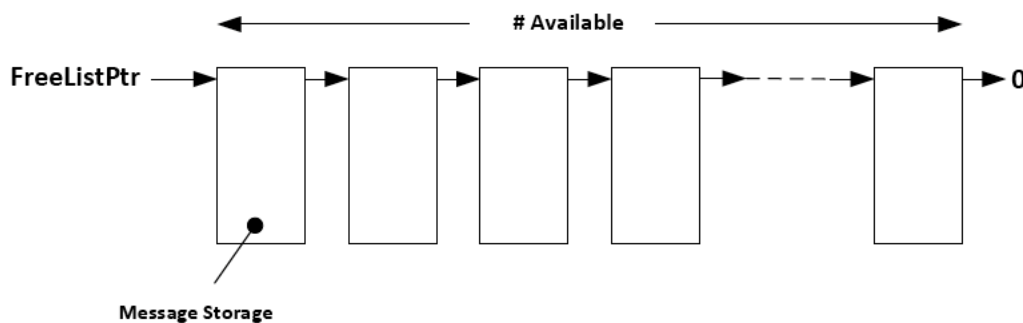
Figure 7.3: Message copying and memory use for sending and receiving messages.

A message queue is a kernel object (i.e., a data structure) through which messages are sent (i.e., posted) from either interrupt service routines (ISRs) or tasks to another task (i.e., pending). An application can have any number of message queues, each one having its own purpose. For example, a message queue can be used to pass packets received from a communication interface ISR to a task, which in turn would be responsible for processing the packet. Another queue can be used to pass content to a display task that will be responsible for properly updating a display.



STORAGE AREA

Messages are typically void pointers to a storage area containing the actual message. However, the pointer can point to anything, even a function for the receiving task to execute. The meaning of the message is thus application-dependent. Each message queue is configurable in the amount of storage it will hold. A message queue can be configured to hold a single message (a.k.a., a mailbox) or N messages. The size of the queue depends on the application and how fast the receiving task can process messages before the queue fills up.



USAGE OF A MESSAGE QUEUE

A message queue is used to send data from an ISR or a task to another task. It is like a "Shoot and forget" operation. The sending task or ISR sends the message and then hopes that another task will receive it later on. The message queue also includes task synchronization in the way that when a task sends a message and if there is a task waiting for messages from the message queue,

that task will also receive the message and then be unblocked. So there is no need to inform the receiving task in another way about the message.

The RTOS does not care about what is the content of the message, it just copies the message into one of its own buffers, and when a task wants to receive the message, then the message is copied to the receiving task buffer. So the message can contain any type of data from the complete set of data to a just a pointer to the data or sometimes just be a dummy message that is simply used to unblock another task.

UNIT – III

OBJECTS, SERVICES AND I/O

PIPES

Pipes are kernel objects that provide unstructured data exchange and facilitate synchronization among tasks. In a traditional implementation, a pipe is a unidirectional data exchange facility, as shown in Figure 8.1. Two descriptors, one for each end of the pipe (one end for reading and one for writing), are returned when the pipe is created. Data is written via one descriptor and read via the other. The data remains in the pipe as an unstructured byte stream. Data is read from the pipe in FIFO order.

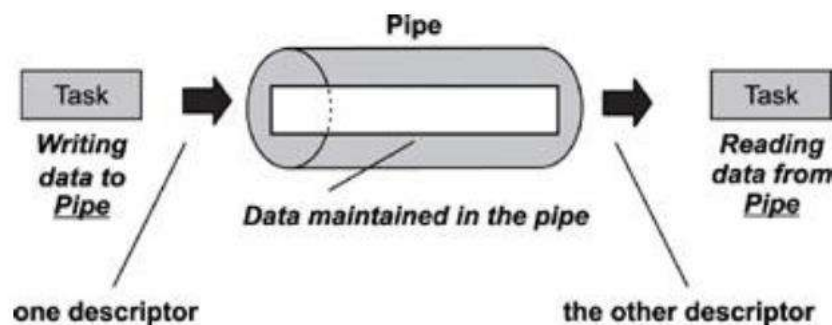
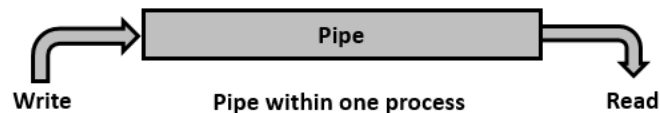


Figure 8.1: A common pipe unidirectional.

A pipe provides a simple data flow facility so that the reader becomes blocked when the pipe is empty, and the writer becomes blocked when the pipe is full. Typically, a pipe is used to exchange data between a data-producing task and a data-consuming task, as shown in Figure 8.2. It is also permissible to have several writers for the pipe with multiple readers on it.

Pipe is a communication medium between two or more related or interrelated processes. It can be either within one process or a communication between the child and the parent processes. Communication can also be multi-level such as communication between the parent, the child and the grand-child, etc. Communication is achieved by one process writing into the pipe and other reading from the pipe. To achieve the pipe system call, create two files, one to write into the file and another to read from the file.

Pipe mechanism can be viewed with a real-time scenario such as filling water with the pipe into some container, say a bucket, and someone retrieving it, say with a mug. The filling process is nothing but writing into the pipe and the reading process is nothing but retrieving from the pipe. This implies that one output (water) is input for the other (bucket).



```
#include<unistd.h>
```

```
int pipe(int pipedes[2]);
```

This system call would create a pipe for one-way communication i.e., it creates two descriptors, first one is connected to read from the pipe and other one is connected to write into the pipe.

Example program: Program to write and read two messages using pipe.

Algorithm:

Step 1: Create a pipe.

Step 2: Send a message to the pipe.

Step 3: Retrieve the message from the pipe and write it to the standard output.

Step 4: Send another message to the pipe.

Step 5: Retrieve the message from the pipe and write it to the standard output.

Two-way Communication Using Pipes: Pipe communication is viewed as only one-way communication i.e., either the parent process writes and the child process reads or vice-versa but not both. However, what if both the parent and the child needs to write and read from the pipes simultaneously, the solution is a two-way communication using pipes. Two pipes are required to establish two-way communication. Following are the steps to achieve two-way communication:

Step 1: Create two pipes. First one is for the parent to write and child to read, say as pipe1. Second one is for the child to write and parent to read, say as pipe2.

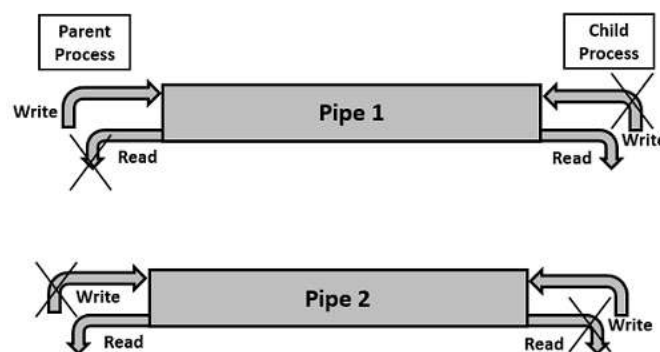
Step 2: Create a child process.

Step 3: Close unwanted ends as only one end is needed for each communication.

Step 4: Close unwanted ends in the parent process, read end of pipe1 and write end of pipe2.

Step 5: Close the unwanted ends in the child process, write end of pipe1 and read end of pipe2.

Step 6: Perform the communication as required.



Sample Programs: Achieving two-way communication using pipes.

Algorithm:

Step 1: Create pipe1 for the parent process to write and the child process to read.

Step 2: Create pipe2 for the child process to write and the parent process to read.

Step 3: Close the unwanted ends of the pipe from the parent and child side.

Step 4: Parent process to write a message and child process to read and display on the screen.

Step 5: Child process to write a message and parent process to read and display on the screen.

EVENT REGISTERS

Some kernels provide a special register as part of each task's control block, as shown in Figure 8.7. This register, called an event register, is an object belonging to a task and consists of a group of binary event flags used to track the occurrence of specific events. Depending on a given kernel's implementation of this mechanism, an event register can be 8-, 16-, or 32-bits wide, maybe even more. Each bit in the event register is treated like a binary flag (also called an event flag) and can be either set or cleared.

Through the event register, a task can check for the presence of particular events that can control its execution. An external source, such as another task or an ISR, can set bits in the event register to inform the task that a particular event has occurred.

Applications define the event associated with an event flag. This definition must be agreed upon between the event sender and receiver using the event register.

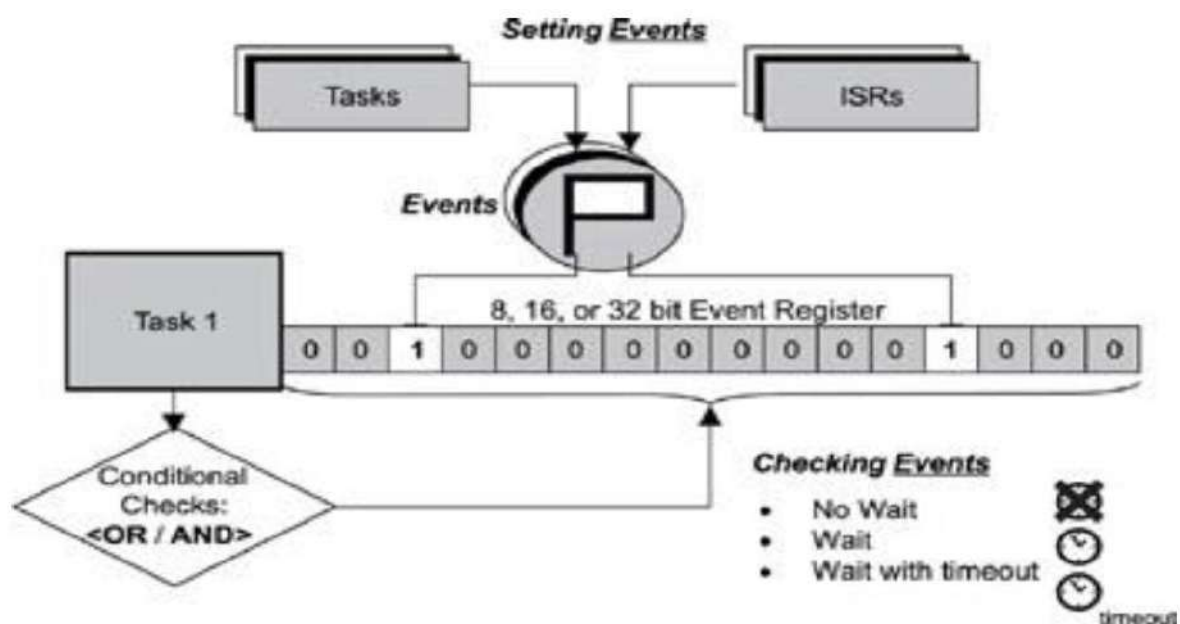


Figure : Event register.

Event Register Control Blocks: Typically, when the underlying kernel supports the event register mechanism, the kernel creates an event register control block as part of the task control block when creating a task, as shown in Figure 8.8.

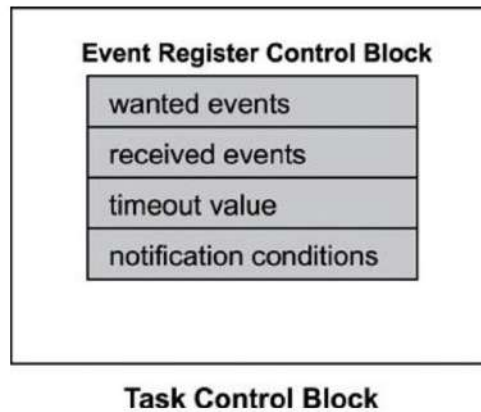


Figure : Event register control block.

The task specifies the set of events it wishes to receive. This set of events is maintained in the wanted events register. Similarly, arrived events are kept in the received events register. The task indicates a timeout to specify how long it wishes to wait for the arrival of certain events. The kernel wakes up the task when this timeout has elapsed if no specified events have arrived at the task.

Using the notification conditions, the task directs the kernel as to when it wishes to be notified (awakened) upon event arrivals. For example, the task can specify the notification conditions as “send notification when both event type 1 and event type 3 arrive or when event type 2 arrives.” This option provides flexibility in defining complex notification patterns.

Typical Event Register Operations: Two main operations are associated with an event register, the sending and the receiving operations, as shown in Table 8.5.

Table : Event register operations.

Operation	Description
Send	Sends events to a task
Receive	Receives events

The receive operation allows the calling task to receive events from external sources. The task can specify if it wishes to wait, as well as the length of time to wait for the arrival of desired events before giving up. The task can wait

forever or for a specified interval. Specifying a set of events when issuing the receive operation allows a task to block-wait for the arrival of multiple events, although events might not necessarily all arrive simultaneously. The kernel translates this event set into the notification conditions. The receive operation returns either when the notification conditions are satisfied or when the timeout has occurred. Any received events that are not indicated in the receive operation are left pending in the received events register of the event register control block. The receive operation returns immediately if the desired events are already pending.

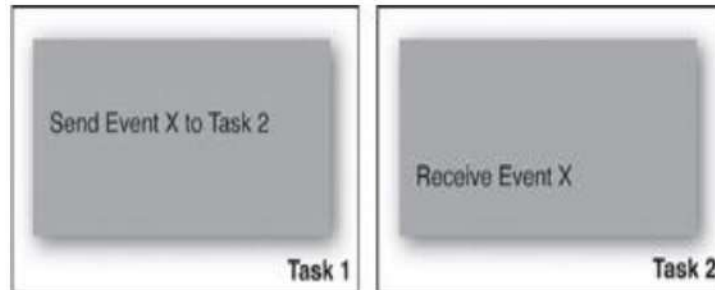
The event set is constructed using the bit-wise AND/OR operation. With the AND operation, the task resumes execution only after every event bit from the set is on. A task can also block-wait for the arrival of a single event from an event set, which is constructed using the bit-wise OR operation. In this case, the task resumes execution when any one event bit from the set is on.

The send operation allows an external source, either a task or an ISR, to send events to another task. The sender can send multiple events to the designated task through a single send operation. Events that have been sent and are pending on the event bits but have not been chosen for reception by the task remain pending in the received events register of the event register control block.

Events in the event register are not queued. An event register cannot count the occurrences of the same event while it is pending; therefore, subsequent occurrences of the same event are lost. For example, if an ISR sends an event to a task and the event is left pending; and later another task sends the same event again to the same task while it is still pending, the first occurrence of the event is lost.

Typical Uses of Event Registers: Event registers are typically used for unidirectional activity synchronization. It is unidirectional because the issuer of the receive operation determines when activity synchronization should take place. Pending events in the event register do not change the execution state of the receiving task.

In following the diagram, at the time task 1 sends the event X to task 2, no effect occurs to the execution state of task 2 if task 2 has not yet attempted to receive the event.



No data is associated with an event when events are sent through the event register. Other mechanisms must be used when data needs to be conveyed along with an event. This lack of associated data can sometimes create difficulties because of the noncumulative nature of events in the event register. Therefore, the event register by itself is an inefficient mechanism if used beyond simple activity synchronization.

Another difficulty in using an event register is that it does not have a built-in mechanism for identifying the source of an event if multiple sources are possible. One way to overcome this problem is for a task to divide the event bits in the event register into subsets.

The task can then associate each subset with a known source. In this way, the task can identify the source of an event if each relative bit position of each subset is assigned to the same event type.

In Figure 8.9, an event register is divided into 4-bit groups. Each group is assigned to a source, regardless of whether it is a task or an ISR. Each bit of the group is assigned to an event type.

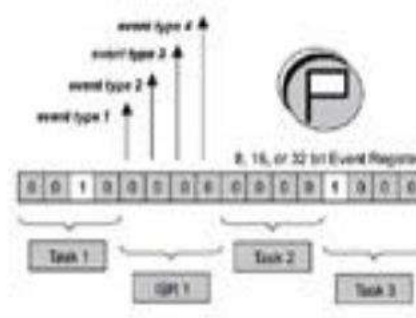
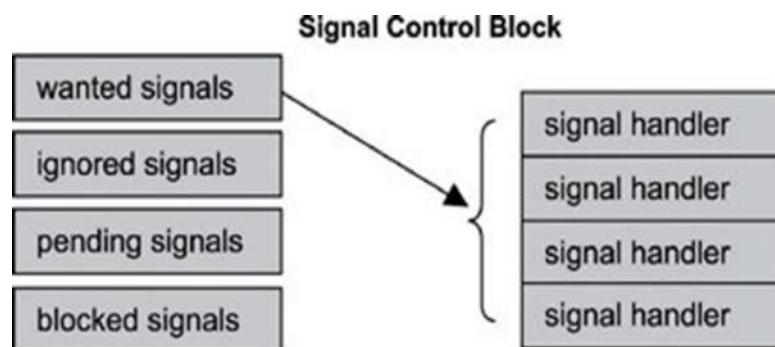


Figure 8.9: Identifying an event source.

SIGNALS

- ⇒ A signal is a software interrupt that is generated when an event has occurred.
- ⇒ It diverts the signal receiver from its normal execution path and triggers the associated asynchronous processing.

Signal Control Blocks:



Task Control Block

- ⇒ The signal control block maintains a set of signals the wanted signals which the task is prepared to handle.
- ⇒ When a task is prepared to handle a signal, it is often said, the task is ready to catch the signal. When a signal interrupts a task, it is often said, the signal is raised to the task.
- ⇒ The task can provide a signal handler for each signal to be processed, or it can execute a default handler that the kernel provides.
- ⇒ It is possible to have a single handler for multiple types of signals.

Typical Signal Operations:

Operation	Description
Catch	Installs a signal handler
Release	Removes a previously installed handler
Send	Sends a signal to another task
Ignore	Prevents a signal from being delivered
Block	Blocks a set of signal from being delivered
Unblock	Unblocks the signals so they can be delivered

Typical Uses of Signals:

- 1) Some signals are associated with hardware events and thus are usually sent by hardware ISRs. The ISR is responsible for immediately responding to these events.
- 2) The ISR, however, might also send a signal so that tasks affected by these hardware events can conduct further, task-specific processing.
- 3) Signals can also be used for synchronization between tasks.
- 4) Using signals can be expensive due to the complexity of the signal facility when used for inter-task synchronization.
- 5) A signal alters the execution state of its destination task. Because signals occur asynchronously, the receiving task becomes nondeterministic, which can be undesirable in real-time system.
- 6) Many implementations do not support queuing or counting of signals. In these implementations, multiple occurrences of the same signal overwrite each other. For example, a signal delivered to a task multiple times before its handler is invoked has the same effect as a single delivery. The task has no way to determine if a signal has arrived multiple times.
- 7) Many implementations do not support signal delivery that carries information, so data cannot be attached to a signal during its generation.

- 8) Many implementations do not support a signal delivery order, and signals of various types are treated as having equal priority, which is not ideal. For example, a signal triggered by a page fault is obviously more important than a signal generated by a task indicating it is about to exit. On an equal-priority system, the page fault might not be handled first.
- 9) Many implementations do not guarantee when an unblocked pending signal will be delivered to the destination task.
- 10) Some kernels do implement real-time extensions to traditional signal handling, which allows.

OTHER BUILDING BLOCKS

These other common building blocks make up the additional kernel services that are part of various embedded applications. The other building blocks include the following:

- 1) TCP/IP protocol stack.
- 2) File system component.
- 3) Remote procedure call (RPC) component.
- 4) Command shell.
- 5) Target debut agent.
- 6) Other components.

1) TCP/IP Protocol Stack: The network protocol stacks and components, as illustrated in Figure 9.2, provide useful system services to an embedded application in a networked environment. The TCP/IP protocol stack provides transport services to both higher layer, well-known protocols, including Simple Network Management Protocol (SNMP), Network File System (NFS), and Telnet, and to user-defined protocols. The transport service can be either reliable connection-oriented service over the TCP protocol or unreliable connectionless service over the UDP protocol. The TCP/IP protocol stack can operate over various types of physical connections and networks, including Ethernet, Frame Relay, ATM, and ISDN networks using different frame encapsulation protocols, including the point-to-point protocol. It is common

to find the transport services offered through standard Berkeley socket interfaces.

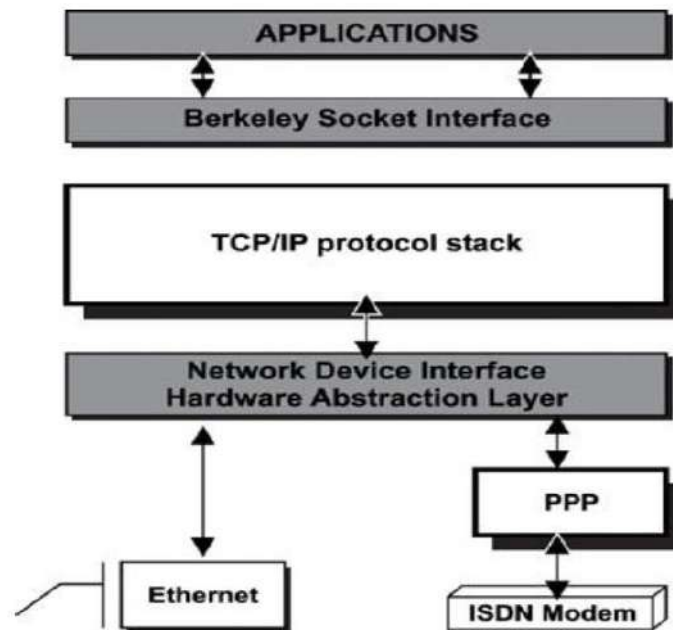


Figure 9.2: TCP/IP protocol stack component.

2) File System Component: The file system component, as illustrated in Figure 9.3, provides efficient access to both local and network mass storage devices. These storage devices include but are not limited to CD-ROM, tape, floppy disk, hard disk, and flash memory devices. The file system component structures the storage device into supported formats for writing information to and for accessing information from the storage device. For example, CD-ROMs are formatted and managed according to ISO 9660 standard file system specifications; floppy disks and hard disks are formatted and managed according to MS-DOS FAT file system conventions and specifications; NFS allows local applications to access files on remote systems as an NFS client. Files located on an NFS server are treated exactly as though they were on a local disk. Because NFS is a protocol, not a file system format, local applications can access any format files supported by the NFS server. File system components found in some real-time RTOS provide high-speed proprietary file systems in place of common storage devices.

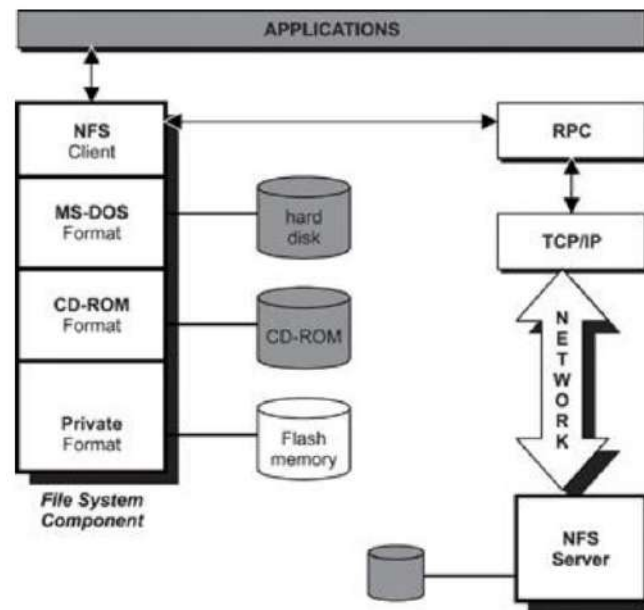


Figure 9.3: File system component.

3) Remote Procedure Call (RPC) Component: The remote procedure call (RPC) component allows for distributed computing. The RPC server offers services to external systems as remotely callable procedures. A remote RPC client can invoke these procedures over the network using the RPC protocol. To use a service provided by an RPC server, a client application calls routines, known as stubs, provided by the RPC client residing on the local machine.

The RPC client in turn invokes remote procedure calls residing in the RPC server on behalf of the calling application. The primary goal of RPC is to make remote procedure calls transparent to applications invoking the local call stubs. To the client application, calling a stub appears no different from calling a local procedure. The RPC client and server can run on top of different operating systems, as well as different types of hardware. As an example of such transparency, note that NFS relies directly upon RPC calls to support the illusion that all files are local to the client machine.

To hide both the server remoteness, as well as platform differences from the client application, data that flows between the two computing systems in the RPC call must be translated to and from a common format. External data representation (XDR) is a method that represents data in an OS- and machine-independent manner. The RPC client translates data passed in as procedure parameters into XDR format before making the remote procedure

call. The RPC server translates the XDR data into machine-specific data format upon receipt of the procedure call request. The decoded data is then passed to the actual procedure to be invoked on the server machine. This procedure's output data is formatted into XDR when returning it to the RPC client. The RPC concept is illustrated in Figure 9.4.

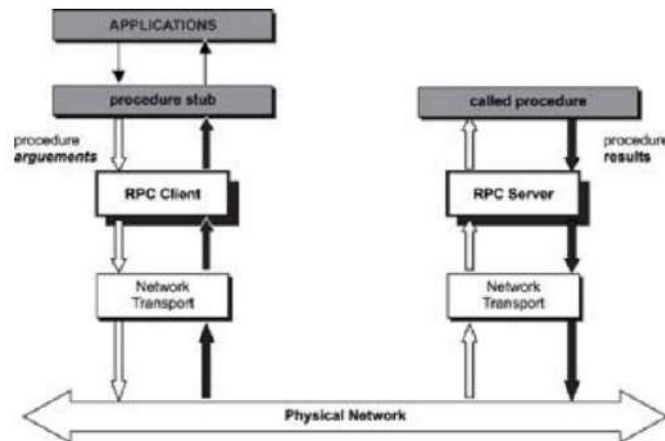


Figure 9.4: Remote procedure calls.

4) Command Shell: The command shell, also called the command interpreter, is an interactive component that provides an interface between the user and the real-time operating system. The user can invoke commands, such as ping, ls, loader, and route through the shell. The shell interprets these commands and makes corresponding calls into RTOS routines. These routines can be in the form of loadable program images, dynamically created programs (dynamic tasks), or direct system function calls if supported by the RTOS. The programmer can experiment with different global system calls if the command shell supports this feature. With this feature, the shell can become a great learning tool for the RTOS in which it executes.

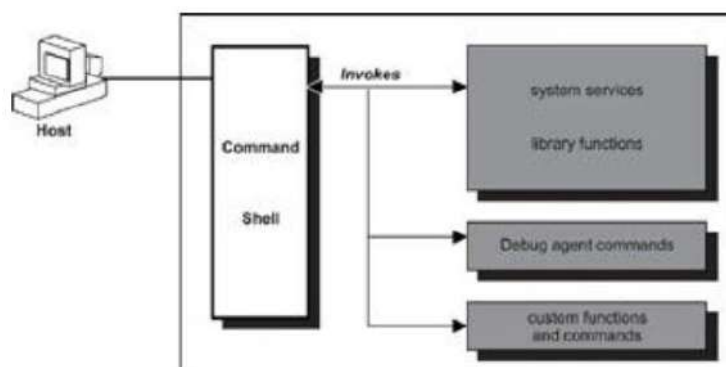


Figure : RTOS command shell.

Some command shell implementations provide a programming interface. A programmer can extend the shell's functionality by writing additional commands or functions using the shell's application program interface (API). The shell is usually accessed from the host system using a terminal emulation program over a serial interface. It is possible to access the shell over the network, but this feature is highly implementation-dependent. The shell becomes a good debugging tool when it supports available debug agent commands. A host debugger is not always available and can be tedious to set up. On the other hand, the programmer can immediately begin debugging when a debug agent is present on the target system, as well as a command shell.

5) Target Debug Agent: Every good RTOS provides a target debug agent. Through either the target shell component or a simple serial connection, the debug agent offers the programmer a rich set of debug commands or capabilities. The debug agent allows the programmer to set up both execution and data access break points. In addition, the programmer can use the debug agent to examine and modify system memory, system registers, and system objects, such as tasks, semaphores, and message queues. The host debugger can provide source-level debug capability by interacting with the target debug agent. With a host debugger, the user can debug the target system without having to understand the native debug agent commands.

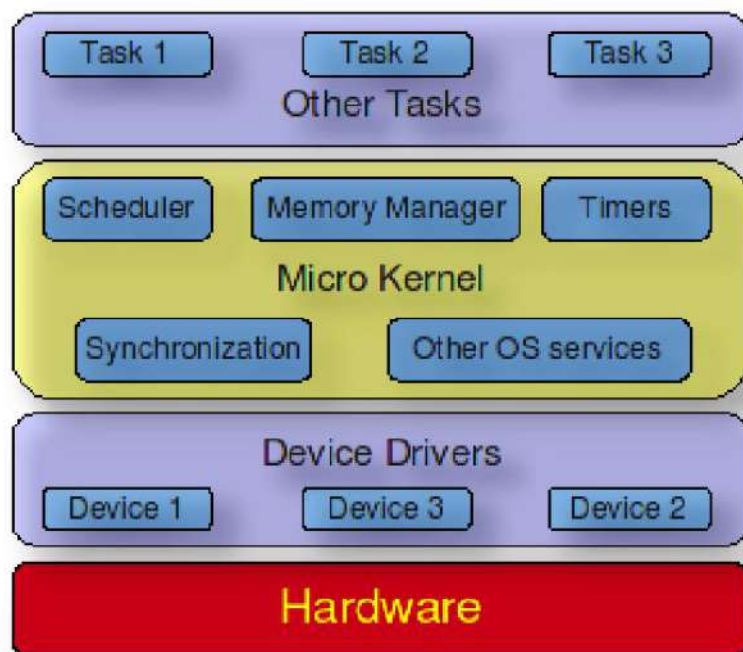
The target debug agent commands are mapped into host debugger commands that are more descriptive and easier to understand. Using an established debug protocol, the host debugger sends the user-issued debug commands to the target debug agent over the serial cable or the Ethernet network. The target debug agent acts on the commands and sends the results back to the host debugger. The host debugger displays the results in its user-friendly debug interface. The debug protocol is specific to the host debugger and its supported debug agent. Be sure to check the host debugging tools against the supported RTOS debug agents before making a purchase.

6) Other Components: What has been presented so far is a very small set of components commonly found in available RTOS. Other service components

include the SNMP component. The target system can be remotely managed over the network by using SNMP. The standard I/O library provides a common interface to write to and read from system I/O devices. The standard system library provides common interfaces to applications for memory functions and string manipulation functions. These library components make it straightforward to port applications written for other operating systems as long as they use standard interfaces.

The possible services components that an RTOS can provide are limited only by imagination. The more an embedded RTOS matures the more components and options it provides to the developer. These components enable powerful embedded applications programming, while at the same time save overall development costs. Therefore, choose the RTOS wisely.

COMPONENT CONFIGURATION OF RTOS



The RTOS component of the real time system is just the micro kernel part, the yellow region. The RTOS just exports a set of basic components as part of the OS, a priority based task scheduler, a minimalistic memory management subsystem, a set of timers, some task synchronization facilities like semaphores and conditional variables and any other such OS services. The timers and synchronization facilities trigger the scheduler almost everytime.

If time slicing is implemented, timer also issues a regular tick which causes currently running tasks to be preempted and next higher priority task to be rescheduled.

BASIC I/O CONCEPTS

The combination of I/O devices, device drivers, and the I/O subsystem comprises the overall I/O system in an embedded environment.

The purpose of the I/O subsystem “To hide the device-specific information from the kernel as well as from the application developer”. To provide a uniform access method to the peripheral I/O devices of the system.

I/O SUBSYSTEM

- ⇒ Each I/O device driver can provide a driver specific set of I/O application programming interfaces to the applications.
- ⇒ However, each application must be aware of the nature of the underlying I/O device.
- ⇒ Thus, embedded systems often include an I/O subsystem to reduce this implementation dependence.
- ⇒ I/O subsystem defines a standard set of functions for I/O operations.
- ⇒ To hide device peculiarities from applications.
- ⇒ All I/O device drivers conform to and support this function set.
- ⇒ To provide uniform I/O to applications across a wide spectrum of I/O devices of varying types.

UNIT – IV

EXCEPTIONS, INTERRUPTS AND TIMERS

EXCEPTIONS

An exception is any event that disrupts the normal execution of the processor and forces the processor into execution of special instructions in a privileged state. Exceptions can be classified into two categories: synchronous exceptions and asynchronous exceptions.

Exceptions raised by internal events, such as events generated by the execution of processor instructions, are called synchronous exceptions. Examples of synchronous exceptions include the following:

- 1) On some processor architectures, the read and the write operations must start at an even memory address for certain data sizes. Read or write operations that begin at an odd memory address cause a memory access error event and raise an exception (called an alignment exception).
- 2) An arithmetic operation that results in a division by zero raises an exception.

Exceptions raised by external events, which are events that do not relate to the execution of processor instructions, are called asynchronous exceptions. In general, these external events are associated with hardware signals. The sources of these hardware signals are typically external hardware devices. Examples of asynchronous exceptions include the following:

- 1) Pushing the reset button on the embedded board triggers an asynchronous exception (called the system reset exception).
- 2) The communications processor module that has become an integral part of many embedded designs is another example of an external device that can raise asynchronous exceptions when it receives data packets.

An interrupt, sometimes called an external interrupt, is an asynchronous exception triggered by an event that an external hardware device generates. Interrupts are one class of exception. What differentiates interrupts from other types of exceptions, or more precisely what differentiates synchronous exceptions from asynchronous exceptions, is the source of the event. The event source for a synchronous exception is internally generated from the processor due to the execution of some instruction. On the other hand, the event source for an asynchronous exception is an external hardware device.

Because the term interrupt has been used extensively in other texts, therefore, the text that follows uses exceptions to mean synchronous exceptions and interrupts to mean asynchronous exceptions. The book uses general exceptions to mean both. The term interrupts and external interrupts are used interchangeably throughout the text.

Exceptions and interrupts are the necessary evils that exist in the majority of embedded systems. This facility, specific to the processor architecture, if misused, can become the source of troubled designs. While exceptions and interrupts introduce challenging design complications and impose strict coding requirements, they are nearly indispensable in embedded applications. The following sections describe the most common and important uses of these mechanisms.

INTERRUPTS

The time a system takes to service an interrupt, that is, the time between an interrupt being asserted and the first instruction of the associated interrupt service routine (ISR) being run, is known as the system's interrupt latency. An RTOS will generally have a far more fastidious attention to reducing this latency than a general OS.

Interrupt disabling inherently increases latency. If interrupts are disabled whilst an event occurs, the event cannot be serviced until interrupts are enabled again. If this event is critical, the latency introduced may be unacceptable.

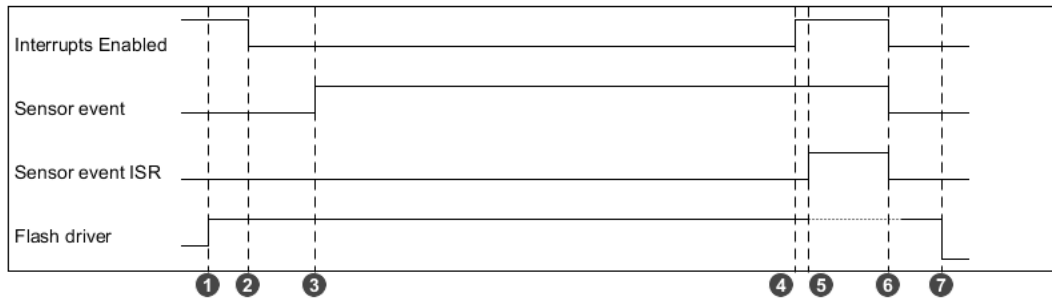
RTOSes are specifically designed to minimise interrupt latency. One of the ways in which they do this is through policy: the RTOS will use atomic primitives where possible to avoid disabling interrupts and will only disable interrupts where absolutely necessary and for as short a time as possible.

Another strategy for reducing latency is to carry out most of the interrupt processing outside of the interrupt context. This is something that PREEMPT-RT introduced to Linux:

“Instead of running interrupt handlers in hard interrupt context, PREEMPT.RT runs them in kernel threads ... The kernel thread becomes interruptible ... you can put a low priority on non-important interrupts and a higher priority on important ... tasks.”

The ISR routine will generally do as little as possible and will leave the real processing of the ISR to a task. The idea is that normally interrupts are not pre-emptible and so the actual ISR code should complete as quickly as possible to minimise system latency. Tasks, however, are pre-emptible, so by putting the bulk of the interrupt processing into a task allows the non-critical sections of an interrupt to be pre-empted by higher priority tasks and interrupts. Even if the hardware supports the nesting of interrupts they cannot be infinitely nested, and so the same applies. Additionally, interrupt or pre-emption disabling will also be strongly discouraged unless absolutely necessary.

We could imagine this in the context of a self-driving car. Storing sensor data is far less important than stopping the car when someone steps out in front of it, and we see below how disabling interrupts in a device driver can affect the system's response to more critical events. The following timing diagram shows what happens to our system when a badly written storage driver, which wants to do an atomic write of a block of data to the device, disables interrupts to ensure that it isn't pre-empted. I have actually seen a driver do this “out in the wild”, although thankfully not in anything that would go anywhere near a safety critical system!



- 1) The storage driver begins running its write routine.
- 2) The storage driver disables interrupts to ensure the write is atomic.
- 3) Meanwhile, the sensor system detects that someone has stepped out in front of the car. The subsystem interrupts the CPU, but is held off because interrupts are still disabled.
- 4) The storage driver re-enables interrupts because it has finished doing its write.
- 5) The OS is now interrupted by the unmasked detection event and the sensors system's ISR is run. What?! Wait a minute! The car has detected that a person has stepped out in front of it, but it waited $t_5 - t_3$ time periods before it did anything about it?! The time defined by $t_{latency} = t_5 - t_3$ is the interrupt latency in this case and I'm sure we'd all agree it is completely unacceptable.

This, of course, is a pretty bad scenario and obviously contrived. First of all the sensor event would never be tied to an interrupt that can be masked and secondly one should never write a driver like this! The principle, however, is clearly shown. The RTOS has to respond to events as promptly as possible to meet its deadlines and to help ensure this, interrupt latency must be kept to a minimum.

Different RTOS designs have different approaches to task vs. interrupt priorities. Green Hill's INTEGRITY RTOS, for example, helps reduce latency by doing fancy things like letting task priorities and interrupt priorities overlap. This is handy because when a critical task is about to run, the RTOS will automatically disable lower priority interrupts when it schedules in the task. This, of course, relies on the hardware interrupt controller and hardware

design being able to support this behaviour too. They also claim that interrupts are never disabled during system calls.

We've learnt about interrupt disabling and the effect it can have on a system. The next article will cover a barrier to correct pre-emption and adherence to priorities that are very particular to real-time systems, namely priority inversion.

APPLICATIONS

A real-time application (RTA) is an application program that functions within a time frame that the user senses as immediate or current. The latency must be less than a defined value, usually measured in seconds. ... The use of RTAs is called real-time computing (RTC).

An article on Embedded System and its Real Time Applications. Focuses on different topics like What is an Embedded System, What are the Real Time Applications of Embedded Systems, What is the Future of Embedded Systems, etc. The World is filled with Embedded Systems.

The development of Microcontroller has paved path for several Embedded System application and they play a significant role (and will continue to play in the future as well) in our modern day life in one way or the other. Starting from consumer electronics like Digital Cameras, DVD Players to high end and advanced systems like Flight Controllers and Missile Guidance Systems, embedded systems are omnipresent and became an important part of our life. The way we live our life has been significantly improved with the utilization of Embedded Systems and they will continue to be an integral part of our lives even tomorrow. Another important concept we are hearing these days is Real – Time Systems. In a real time system,

Real Time Computing takes place, where a computer (an Embedded System) must generate response to events within certain time limits. Before going in to the details of Real Time Applications of Embedded Systems, we will first see what an Embedded System is, what is a real time system and what is real time operating system.

UNIT – V**CASE STUDIES OF RTOS****VX WORKS (WIND RIVER SYSTEMS)**

VxWorks is the premier development and execution environment for complex real-time and embedded applications on a wide variety of target processors. Three highly integrated components are included with VxWorks: a high performance scalable real-time operating system which executes on a target processor; a set of powerful cross-development tools; and a full range of communications software options such as Ethernet or serial line for the target connection to the host. The heart of the OS is the Wind microkernel which supports multitasking, scheduling, intertask management and memory management. All other functionalities are through processes. There is no privilege protection between system and application and also the support for communication between processes on different processors is poor.

Prepared By:

RIYAZ MOHAMMED